

AI Driven Business Sales Forecasting System



Session: BSCS Fall 2022

Project Advisor: Ms. Sawera Kanwal

Submitted By

Name	Roll No	Program	Contact	Email
Muhammad Umair Bilal	B-27430	CS	03000680158	b-27430@student.usa.edu.pk
Hafiz Mudasir Bilal	B-27431	CS	03069686492	b-27431@student.usa.edu.pk
Khuzaima Asif	B-27463	CS	03264699196	b-27463@student.usa.edu.pk

University of South Asia
Department of Computer Science

STATEMENT OF SUBMISSION

This is to certify that **Muhammad Umair Bilal** Roll No. **B-27430**, **Hafiz Mudassir Bilal** Roll No. **B-27431**, and **Khuzaima Asif** Roll No. **B-27463** have successfully completed the final project named as: **AI Driven Business Sales Forecasting System**, at the University of South Asia to fulfill the partial requirement of the degree of **Bachelors in Computer Science**.

PROJECT SUPERVISOR

Name: Ms. Sawera Kanwal

Date: _____

Signature: _____

DEAN OF DEPARTMENT

Name: Dr. Nadeem Asif
(Computer science)

Date: _____

Signature: _____

PROJECT CONVENER

Name: Zaid Bin Javaid

Date: _____

Signature: _____

EXTERNAL

Name: _____

Date: _____

Signature: _____

CERTIFICATE OF ORIGINALITY

We hereby declare that this final year project report titled "**AI Driven Business Sales Forecasting System**" submitted to the Department of Computer Science, University of South Asia, is our own original work. This report has not been submitted anywhere else for any other degree or certification. All sources of information have been duly acknowledged and referenced within the document.

We further declare that the software, code, models, and documentation presented in this project are developed by us under the supervision and guidance of our assigned project advisor. Any material referenced from external sources has been appropriately cited as per the university guidelines.

Name: Muhammad Umair Bilal
Father Name: Muhammad Bilal
Program: Computer Science
Roll no: B-27430

Signature _____

CNIC: _____

Thumb _____

Name: Hafiz Mudassir Bilal
Father Name: Muhammad Bilal
Program: Computer Science
Roll no: B-27431

Signature _____

CNIC: _____

Thumb _____

Name: Khuzaima Asif
Father Name: Muhammad Asif
Program: Computer Science
Roll no: B-27463

Signature _____

CNIC: _____

Thumb _____

Proofreading Certificate

I am writing to confirm that I have read the material very carefully. I am sure that the final project does not have any spelling, punctuation, or grammar issues. As much as was expected from kids who were graduating from college. Overall, I think this document is well-organized.

Name: _____

Signature: _____

Acknowledgement

First of all, we want to thank God for providing us the strength, wisdom, and patience to finish this job successfully. None of this would have been possible without His grace.

We want to thank our Project Advisor, **Ms. Sawera Kanwal**, from the bottom of our hearts for always being there for us, giving us expert advice, and encouraging us throughout the project. His extensive expertise in machine learning and software development enabled us to overcome several obstacles and guide the project towards success. Every time I got comments from him, I learned something new.

We also like to thank the University of South Asia's Department of Computer Science for giving us the tools and academic setting we needed to do this job. We would like to thank all of our teachers who taught us the theory and practice that this project is based on during the last four years.

Finally, we can't thank our families and friends enough for their quiet support and patience, which kept us going through long nights of writing and debugging. This milestone wouldn't have been achievable without each and every one of you.

Abstract

Accurate sales forecasting is now an important part of a long-term business plan in the fast-paced and highly competitive world of retail. Small and medium-sized businesses, in particular, typically don't have access to advanced analytics tools, so they have to make business decisions based on gut feelings and simple spreadsheets. This makes it hard to keep track of inventories, lose out on sales, and make customers unhappy.

This project introduces the AI Driven Business Sales Forecasting System, a web application that helps business owners and retail managers turn their raw transactional data into useful insights that they can use to make decisions. The system has a Streamlit-based front end and two main machine learning models that power it: a time-series sales forecast model and a product recommendation engine.

Users can easily and intuitively upload their past store data. The system first does automated feature engineering and exploratory data analysis, which gives a full set of Key Performance Indicators (KPIs). After that, the combined AI models make projections about the next month's profits and sales for the five most profitable products and the five least profitable or most risky products. The recommendation engine also finds the top five goods that are most often bought together with a certain product. This makes it possible to use upselling and cross-selling methods.

Both models have been trained and tested on a dataset of about 50,000 transactional records, and they both get 80 to 85 percent of the answers right. There are also a number of interactive visualizations in the system, such as graphs of sales trends, profit distributions, and product performance. Users can save and export results, which gives them a permanent record of their analysis that they can look back on later.

The project fills a real gap in the industry and provides a useful, scalable, and easy-to-use way for businesses to make decisions based on data, without requiring any technical understanding from the end user.

Table of Contents

List of Figures	x
List of Tables	xi
Chapter 1: Introduction	1
1.1 Background and Motivation	2
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Project	3
1.5 Project Overview	4
1.6 Report Organization	5
Chapter 2: Literature Review	6
2.1 Overview of Sales Forecasting	7
2.2 Machine Learning in Business Analytics	7
2.2.1 Feature Engineering for Sales Forecasting	8
2.3 Recommendation Systems in Retail	8
2.4 Existing Systems and Tools	8
2.4.1 Microsoft Power BI	9
2.4.2 Tableau	9
2.4.3 Google Looker Studio	9
2.4.4 Python Open-Source Libraries	10
2.5 Gaps in Existing Literature and Tools	10
Chapter 3: Requirements and Specification.....	11
3.1 Functional Requirements	12
1. Data Upload and Input Management	12
2. Data Processing and Feature Engineering	12
3. KPI Dashboard	12
4. Sales Forecasting Model	12
5. Recommendation System	13
6. Visualization and Output	13
3.2 Non-Functional Requirements	14
3.3 Use Case Descriptions	15
Use Case 1: Upload Store Data	15
Use Case 2: View Sales Forecasts	15
Chapter 4: Design	18
4.1 System Architecture.....	18
4.2 Software Architecture and Layer Definitions	18
4.2.1 Presentation Layer (Front-End / HCI)	19

4.2.2 Application Logic Layer (Business Logic / Backend)	19
4.2.3 Data Handling Layer	19
4.2.4 Model Layer	20
4.2.5 Client-Server Architecture	20
4.3 4+1 Architectural View and Design	21
4.3.1 Logical View	21
4.3.2 Process View	22
4.3.3 Development View	22
4.3.4 Physical View	22
4.3.5 Use Case View (+1)	23
4.4 SDLC Model.....	23
4.5 Database Design.....	23
4.5.1 Forecast Output Schema	24
4.5.2 Recommendation Output Schema	25
Chapter 5: Implementation	29
5.1 Technologies and Tools Used.....	27
5.2 Feature Engineering Pipeline.....	27
5.2.1 Data Cleaning	28
5.2.2 Derived Column Creation	28
5.2.3 Aggregation and Product-Level Features	29
5.3 Model Development.....	29
5.3.1 Sales Forecasting Model	29
5.3.2 Product Recommendation Model	30
5.4 Front-End Implementation.....	30
5.5 Output Features	31
Chapter 6: Testing	36
6.1 Testing Strategy	33
6.2 Unit Testing	33
6.2.1 Data Preprocessor Tests	33
6.2.2 Feature Engineering Tests	34
6.3 Integration Testing	34
6.4 User Acceptance Testing	35
6.4.1 UAT Tasks	35
6.4.2 UAT Results	35
Chapter 7: Results and Discussion.....	41
7.1 KPI Dashboard Results	38
7.2 Forecasting Model Results.....	38

7.2.1 Top-5 Most Profitable Products (Sample Output)	39
7.2.2 Top-5 Least Profitable Products (Sample Output)	39
7.3 Recommendation Model Results	40
7.3.1 Sample Recommendation Output	40
7.4 Discussion	41
Chapter 8: Conclusion and Future Work.....	45
8.1 Conclusion	44
8.2 Future Work	44
8.2.1 Real-Time Data Integration	44
8.2.2 Automated Model Retraining	45
8.2.3 Multi-User Authentication and Role-Based Access	45
8.2.4 Deeper Forecasting Granularity	45
8.2.5 Advanced Recommendation Models	45
8.2.6 External Factor Integration	45
8.2.7 Mobile Application	45
References	46
Appendix A: Sample Input Dataset Format.....	47
Appendix B: Installation and Deployment Guide.....	48
B.1 Prerequisites	48
B.2 Local Installation Steps	48
B.3 Cloud Deployment	48
B.4 Configuration Options	49
Appendix C: User Manual	50
C.1 Getting Started	50
C.2 Uploading Your Data	50
C.3 Viewing the KPI Dashboard	50
C.4 Reviewing Sales Forecasts	50
C.5 Using the Recommendation Panel	51
C.6 Exporting Results	51
Appendix D: Testing Artefacts	52
D.1 Unit Test Case Specifications	52
D.2 UAT Session Script	52
D.3 UAT Participant Feedback Summary	53
Appendix E: Key Code Listings	55
E.1 Feature Engineering Pipeline (feature_engineer.py)	55
E.2 XGBoost Model Training (forecasting_model.py)	55
E.3 Apriori Recommendation Engine (recommendation_model.py)	56

Appendix F: Project Risk Register	60
Appendix G: Glossary of Terms	61
Appendix H: Project Risk Register	61
Appendix I: Project Risk Register	63
Appendix J: Project Risk Register	64
Appendix K: Project Risk Register	66
Appendix L: Project Risk Register	69
Appendix M: Project Risk Register	72
Appendix N: Project Risk Register	74

List of Figures

Figure 1: Sequence Diagram	14
Figure 2: Use Case Diagram	16
Figure 3: System Architecture Overview	18
Figure 4: System Architecture (3 layers)	211
Figure 5: Data Flow Diagram	255
Figure 6: Feature Engineering Pipeline	218
Figure 7: Sales Forecast	210
Figure 8: Frontend of the main dashboard of the app	38
Figure 9: Business Overview KPIs	38
Figure 10: Showing Top 5 Profitable and Loss making Products Predictions	41
Figure 11: Showing top 5 recommendations of the products	41
Figure 12: Importing all the necessary libraries	42
Figure 13: Model training pipeline	42

List of Tables

Table 1: Comparison of Existing Tools with Proposed System	10
Table 2: Non-Functional Requirements	15
Table 3: Use Case UC-01 - Upload Store Data	15
Table 4: Use Case UC-02 - View Sales Forecasts	16
Table 5: Forecast Output Schema	24
Table 6: Recommendation Output Schema	25
Table 7: Technologies and Libraries Used	27
Table 8: Data Preprocessor Unit Test Cases	34
Table 9: Feature Engineering Unit Test Cases	34
Table 10: Integration Test Cases	35
Table 11: User Acceptance Testing Results Summary	36
Table 12: Forecasting Model Performance Metrics	39
Table 13: Sample Output - Top-5 Most Profitable Products	39
Table 14: Sample Output - Top-5 Least Profitable Products	40
Table 15: Sample Recommendation Output	41
Table 16: Required Input File Schema	47
Table 17: Application Configuration Parameters	49
Table 18: Sample Unit Test Case Specifications	52
Table 19: UAT Feedback Summary	54
Table 20: Project Risk Register	60
Table 21: Benchmarking Hardware Configurations	70
Table 22: Processing Times by Dataset Size (Recommended Hardware)	71
Table 23: Peak Memory Utilisation at Each Processing Stage (50K rows)	71
Table 24: Competitive Positioning Assessment	75
Table 25: Path to Commercialisation - Additional Development Areas	76
Table 26: Student 1 Primary Contributions	79
Table 27: Student 2 Primary Contributions	80

Chapter 1

Introduction

1.1 Background and Motivation

The retail business has always needed precise sales data to work. For a long time, companies used hand-written sales records and the knowledge of experienced managers to plan their inventory, personnel, and buying. This method worked really well when the market was stable, but the retail world nowadays is a lot more dynamic. Consumer preferences change quickly, seasonal changes cause demand spikes and dips, and the huge amount of transactional data created every day makes it hard to understand by hand and easy to make mistakes.

Today, digital point-of-sale systems and e-commerce platforms generate a large amount of structured sales data for businesses. However, most companies fail to properly understand or utilize this data, which results in a significant missed opportunity.

Research consistently shows that companies that adopt data-driven decision-making outperform their competitors. They achieve higher revenue growth, manage inventory more efficiently, and maintain better customer satisfaction.

Artificial intelligence and machine learning have now matured to a point where they can be reliably used for sales forecasting, even with limited computing resources. Algorithms such as Random Forests, Gradient Boosting, and various regression techniques have proven to deliver highly accurate predictions across retail datasets of different sizes and structures. At the same time, techniques like collaborative filtering and association rule mining have become essential components of modern recommendation systems used by major companies like Amazon and Netflix.

Despite this progress, most small and medium-sized businesses (SMEs) have not been able to take advantage of these technologies. Enterprise-level analytics platforms are often expensive, complex to implement, and require dedicated data science teams to maintain. The purpose of this initiative is to make intelligent sales analytics accessible to everyone. The aim is to develop a system that any business owner can use easily, regardless of their technical background, simply by uploading their shop data in a standard format.

Pakistan's retail industry, which is still mostly tiny and informal, is a great example of how this could work. As digitalization spreads through mobile payments and point-of-sale (POS) systems, more and more raw data is becoming available. However, the tools to make sense of it are still lacking. A forecasting and recommendation system that is easy to use and driven by AI might make a big difference for thousands of small stores across the country.

1.2 Problem Statement

Small and medium-sized retail businesses usually do not have access to tools that allow them to deeply understand their sales data. While software like Microsoft Excel can handle basic calculations and charts, it is not sufficient for advanced tasks like predictive analytics or discovering hidden patterns. As a result, business owners often struggle with key

questions such as: which products will perform best next month, which ones are reducing overall profitability, and what items customers tend to buy together.

If answered correctly, these questions can directly impact important business decisions like inventory planning, marketing strategies, and store layout optimization. However, finding these answers requires applying machine learning models to historical sales data, which is usually only possible for businesses that have dedicated data science teams or the budget to hire external experts.

Even for businesses that have some level of analytics capability, there is often a gap between raw model outputs and meaningful business insights. For example, predicting that a product will generate a certain profit next month is not very useful unless it is supported with context such as key performance indicators, historical trends, and clear, actionable recommendations. Most existing open-source tools fail to effectively bridge this gap.

This project aims to solve the issue of limited access to sales intelligence for SMEs by developing a complete, end-to-end application. The system will take raw store data as input and convert it into structured, easy-to-understand, and visually rich insights, all without requiring any technical expertise from the user.

1.3 Objectives

The primary objectives of this project are as follows:

- To design and develop a user-friendly, web-based application that accepts raw store data uploads from non-technical business users.
- To implement automated feature engineering and exploratory data analysis pipelines that process uploaded data and extract meaningful variables for modelling.
- To develop a machine learning model capable of predicting the next month's sales profit and quantity for individual products with an accuracy of at least 90 percent.
- To develop a product recommendation model that identifies top co-purchased products for any given item in the store's inventory, enabling cross-selling strategies.
- To surface the top five most profitable and top five least profitable products along with their next-month predictions, giving business owners a clear picture of their product portfolio performance.
- To provide interactive, visually appealing graphical output including sales trend lines, profit distribution charts, and product performance comparisons.
- To implement output saving and export functionality so that predictions and analysis results can be retained and revisited.
- To validate both models against a dataset of approximately 50,000 transactional records and achieve a minimum model accuracy of 80 percent on held-out test data.

1.4 Scope of the Project

The AI Driven Business Sales Forecasting System is designed specifically for retail businesses that maintain structured transactional data. The system currently supports

uploaded datasets in standard tabular formats such as CSV or Excel files, containing fields related to product identifiers, sales quantities, revenue, costs, and dates.

Within the scope of this project, the following functionalities are fully included:

- Automated data loading, cleaning, and preprocessing.
- Feature engineering including temporal, categorical, and aggregation-based features.
- KPI computation and dashboard display.
- Product-level next-month profit and quantity forecasting.
- Top-five most profitable product identification and predictions.
- Top-five least profitable product identification and predictions.
- Top-five product recommendations per selected product.
- Graphical visualization of sales trends, product profitability, and model outputs.
- Result saving and display in structured output format.

The following are explicitly outside the current scope of the system:

- Real-time data streaming or live database integration.
- Multi-user authentication and role-based access control.
- Automated model retraining on new data without developer intervention.
- Integration with third-party point-of-sale or ERP systems.
- Forecasting at granularities finer than monthly (e.g., daily or weekly predictions).

1.5 Project Overview

The AI-driven Business Sales Forecasting System is a web application built in Python using Streamlit. It follows a clear, step-by-step workflow that starts with data upload and ends with a detailed analytical dashboard.

When the user opens the application, they see a simple and clean interface to upload their data. After the data is uploaded, the system automatically processes it using preprocessing and feature engineering techniques. It then calculates and displays important Key Performance Indicators (KPIs), including total revenue, total profit, best-performing time periods, and overall profit margins.

After the KPI section, the system runs two main AI models. The first is a supervised regression model that predicts next month's sales at the product level. Based on these predictions, the system highlights the top five most profitable and the top five least profitable products, along with their expected profit and sales quantity.

The second model uses techniques like association rule mining and collaborative filtering to generate product recommendations. It identifies which products are commonly purchased together and suggests relevant items based on the user's selection.

The lower portion of the application displays a series of interactive charts and graphs derived from both the raw data and the model outputs. Users can also save their results, and the system supports output in both tabular and diagrammatic forms.

1.6 Report Organization

The remainder of this report is organized into seven additional chapters as outlined below:

Chapter 2 presents a comprehensive literature review covering prior work in sales forecasting, machine learning applications in retail, product recommendation systems, and existing tools in the market. The chapter identifies gaps in current approaches that this project seeks to fill.

Chapter 3 details the requirements and specifications of the system, including both functional and non-functional requirements. Requirements are organized hierarchically and presented in tabular format for clarity.

Chapter 4 covers the design of the system, including the high-level software architecture, layer definitions, the 4+1 architectural view, class diagrams, the selected SDLC model, and the database schema.

Chapter 5 describes the implementation of the system, detailing the technologies, tools, and frameworks used. It covers the feature engineering pipeline, model development methodology, front-end implementation, and output mechanisms.

Chapter 6 presents the testing strategy and results, including unit testing, integration testing, and user acceptance testing findings.

Chapter 7 discusses the results produced by the system, analyzing model performance metrics, KPI accuracy, recommendation quality, and overall system behavior.

Chapter 8 concludes the report by summarizing key findings and outlining directions for future work and system enhancements.

Chapter 2

Literature Review

2.1 Overview of Sales Forecasting

Sales forecasting is the process of predicting future sales over a specific time period, usually at the product, category, or store level. Accurate forecasting plays a critical role in supply chain management, financial planning, workforce scheduling, and promotional strategies. Over time, this field has evolved from basic statistical techniques to more advanced machine learning approaches.

In the early stages, sales forecasting relied on simple methods like moving averages and exponential smoothing. These techniques assume that future sales are mainly based on past data, with recent observations given more importance. While they are easy to use and understand, they struggle to capture complex patterns such as seasonality, non-linear trends, and the impact of external factors like promotions, holidays, or economic conditions.

Later, more advanced statistical models such as ARIMA (Auto Regressive Integrated Moving Average) and its seasonal version SARIMA were introduced. These models improved forecasting by handling time-based patterns like trends and seasonal cycles. However, they come with limitations, including the need for careful parameter tuning, assumptions like data stationarity, and difficulty in incorporating external variables without additional modifications.

With the rise of machine learning in the early 2000s, forecasting methods became much more powerful. Techniques such as decision trees, support vector regression, and neural networks can capture complex, non-linear relationships between input data and sales outcomes. They also allow the use of a wide range of features, including categorical data, calendar-based variables, past sales (lag features), and rolling statistics. This flexibility gives machine learning models a strong advantage over traditional statistical methods in real-world applications.

2.2 Machine Learning in Business Analytics

Machine learning has become one of the most transformative technologies in business analytics over the past decade. Its applications span from customer churn prediction and fraud detection to demand forecasting and pricing optimization. Within the context of retail sales forecasting, several machine learning paradigms have been explored extensively.

Chen and Guestrin (2016) introduced XGBoost, a gradient boosting framework that quickly became one of the most widely used algorithms in tabular data competitions and real-world forecasting tasks. XGBoost builds an ensemble of decision trees in a sequential manner, with each tree correcting the errors of the previous one. Its built-in regularization, handling of missing values, and computational efficiency made it particularly well-suited to retail datasets with diverse feature types and significant noise.

Breiman (2001) introduced Random Forests, which build multiple decision trees using bootstrapped samples of the training data and random feature subsets, combining their outputs through averaging. Random Forests are robust to overfitting, relatively insensitive

to hyperparameter tuning, and can naturally provide feature importance scores, which makes them highly interpretable in a business context.

Deep learning approaches, particularly Long Short-Term Memory (LSTM) networks and Temporal Convolutional Networks (TCNs), have also been applied to sales forecasting. Salinas et al. (2020) proposed DeepAR, a probabilistic model based on autoregressive recurrent networks trained jointly across many related time series, which became a popular baseline in large-scale retail forecasting benchmarks. However, deep learning models generally require large volumes of data, longer training times, and more careful hyperparameter tuning, which can make them impractical for smaller retail datasets.

For the dataset size of approximately 50,000 records used in this project, ensemble tree-based methods represent the most pragmatic choice. They deliver strong predictive performance without the computational overhead of deep learning, are interpretable enough for business stakeholders, and generalize well even when the feature space is moderately sized.

2.2.1 Feature Engineering for Sales Forecasting

Feature engineering is one of the most important steps in building accurate machine learning models, especially for tabular and time-series data. In retail forecasting tasks, it is often considered more impactful than simply choosing a complex model.

Temporal features derived from the sale date play a key role in improving model performance. These include variables such as day of the week, week of the month, month, quarter, and year. They help capture repeating patterns in customer behavior, like weekly habits and seasonal trends.

Lag features are equally important, as they represent past sales values from previous time periods. These features allow the model to learn dependencies over time, showing how past demand influences future sales. Additionally, rolling statistics such as 7-day, 30-day, and 90-day averages or standard deviations help capture trends and variations at different time scales.

Handling categorical data is another essential part of feature engineering. Variables like product IDs, store IDs, and product categories need to be encoded properly. Techniques such as target encoding, which replaces categories with their average sales values, are particularly effective when dealing with high-cardinality data. This approach helps the model better understand relationships between different products and their sales patterns.

2.3 Recommendation Systems in Retail

Recommendation systems have become a foundational technology in the modern retail and e-commerce ecosystem. Their primary purpose is to surface products or content that a user is likely to be interested in, based on their past behavior or the behavior of similar users. In the physical retail context, recommendation systems are typically deployed to support cross-selling and upselling strategies.

The three major paradigms in recommendation system design are collaborative filtering, content-based filtering, and hybrid approaches. Collaborative filtering makes recommendations based on patterns of co-occurrence or similarity across users and items. In the context of market basket analysis, association rule mining is one of the most widely

used techniques, especially the Apriori algorithm introduced by Rakesh Agrawal and Ramakrishnan Srikant. This method identifies items that are frequently bought together and expresses them as simple rules, such as: if a customer buys item A, they are likely to also buy item B.

Later, research by Badrul Sarwar and his team showed that item-based collaborative filtering is more scalable and often more accurate than user-based methods, especially for large product catalogs. This idea became the foundation for recommendation systems used by companies like Amazon.

Another approach, content-based filtering, focuses on recommending products based on their attributes rather than user behavior. For example, it may suggest items similar to a previously purchased product based on category, price, or brand. While this method is easy to understand, it depends heavily on having complete and high-quality product data.

For this project, association rule mining was chosen as the core recommendation technique. Since the dataset consists of transactional records instead of user ratings, this approach is the most suitable and practical. The system analyzes which products are frequently purchased together and then recommends the top five related items for any selected product, making the results both simple and useful for business decision-making.

2.4 Existing Systems and Tools

A number of commercial and open-source tools exist in the sales analytics and forecasting space, and a review of their strengths and limitations was conducted to position this project relative to the existing landscape.

2.4.1 Microsoft Power BI

Power BI is a business intelligence platform developed by Microsoft that offers a wide range of data visualization and basic analytical capabilities. It allows users to create interactive dashboards from structured data sources and supports a variety of connectors to databases, spreadsheets, and online services. Power BI has a relatively gentle learning curve for business users and is widely adopted in enterprise settings.

However, Power BI's native forecasting capabilities are limited to simple linear and exponential smoothing models embedded within its chart components. For more sophisticated machine learning-based forecasting, users must integrate Power BI with external Python or R scripts, which significantly raises the technical barrier. Furthermore, its recommendation engine functionality is absent entirely without custom development.

2.4.2 Tableau

Tableau is another leading data visualization platform that offers powerful charting and dashboard capabilities. Like Power BI, it supports basic trend forecasting but does not natively support machine learning models or recommendation systems. Its Tableau Prep feature adds some data preparation functionality, but the platform is primarily designed for visualization rather than predictive analytics.

2.4.3 Google Looker Studio

Google Looker Studio (formerly Data Studio) is a free, cloud-based visualization tool that connects to Google Sheets, BigQuery, and other data sources. While it is accessible and

capable for basic reporting, it offers no forecasting or recommendation capabilities whatsoever.

2.4.4 Python Open-Source Libraries

Libraries such as Scikit-learn, XGBoost, LightGBM, and Prophet provide powerful forecasting capabilities to developers with Python skills. These tools are state-of-the-art in terms of model performance but present a very high barrier to entry for non-technical users. They require knowledge of Python programming, data preprocessing, model evaluation, and visualization coding, all of which are skills that the typical small business owner does not possess.

2.5 Gaps in Existing Literature and Tools

The review of existing research and commercial tools highlights a clear gap: there are very few end-to-end solutions that combine automated feature engineering, machine learning-based sales forecasting, and product recommendations within a single, easy-to-use interface for non-technical users.

Most commercial business intelligence tools focus mainly on data visualization and historical reporting rather than predictive analytics. Even when forecasting features are available, they are usually limited to basic statistical methods. On the other hand, advanced machine learning tools and libraries require strong technical expertise, making them inaccessible for many businesses. None of the existing solutions fully integrate next-month product-level profit forecasting with a built-in recommendation system in a simple, data-upload-based application.

This gap forms the core motivation for this project. The AI Driven Business Sales Forecasting System is specifically designed to fill this space by providing machine learning-powered forecasting and product recommendations through a straightforward interface, allowing business owners to use it easily without any technical knowledge.

Tool System	/ Pricing	Launch Year	Forecasting	Recommendations	Non-Technical UI
Power BI	\$14.00 user/month, paid yearly	July 2015	Basic only	No	Yes
Tableau	\$15 USD/User/Month	2003	Basic only	No	Yes
Google Looker	Annual commitment	January 2012	No	No	Yes
Proposed System	Free to use	2026 (Latest)	Advanced ML	Yes (Top-5)	Yes (Easy UI)

Table 1: Comparison of Existing Tools with Proposed System

Chapter 3

Requirements and Specification

3.1 Functional Requirements

Functional requirements describe the specific behaviors and functionalities the system must provide. They define what the system does from the user's perspective. The following functional requirements have been identified and organized hierarchically:

1. Data Upload and Input Management

1.1 File Upload

- 1.1.1 The system shall provide a file upload interface that accepts CSV and Excel (.xlsx) file formats.
- 1.1.2 The system shall validate the uploaded file for correct structure and required column presence.
- 1.1.3 The system shall provide informative error messages if the uploaded file is malformed or missing required columns.

1.2 Data Preview

- 1.2.1 The system shall display a tabular preview of the uploaded data for user verification.

2. Data Processing and Feature Engineering

2.1 Preprocessing

- 2.1.1 The system shall automatically handle missing values in the uploaded dataset.
- 2.1.2 The system shall detect and remove duplicate transactional records.
- 2.1.3 The system shall parse and standardize date fields from uploaded data.

2.2 Feature Engineering

- 2.2.1 The system shall generate temporal features including month, quarter, day of week, and year from date fields.
- 2.2.2 The system shall compute lag features for sales quantity and profit at defined lag intervals.
- 2.2.3 The system shall compute rolling mean and rolling standard deviation features at 7, 30, and 90-day windows.

3. KPI Dashboard

3.1 Key Performance Indicators

- 3.1.1 The system shall compute and display total revenue, total cost, and total profit for the uploaded dataset.
- 3.1.2 The system shall compute and display total number of transactions, unique products sold, and average order value.
- 3.1.3 The system shall compute and display the best-performing month and year by total profit.
- 3.1.4 The system shall compute and display overall profit margin percentage.

4. Sales Forecasting Model

4.1 Model Execution

4.1.1 The system shall apply the trained forecasting model to the processed data to generate next-month predictions.

4.1.2 The system shall generate next-month profit and quantity predictions for each product in the dataset.

4.2 Top-5 Most Profitable Products

4.2.1 The system shall identify and display the top five products by predicted next-month profit.

4.2.2 For each of the top five most profitable products, the system shall display the product name, predicted profit, and predicted quantity.

4.3 Top-5 Least Profitable Products

4.3.1 The system shall identify and display the five products predicted to generate the lowest profit in the next month.

4.3.2 For each of the top five least profitable products, the system shall display the product name, predicted profit, and predicted quantity.

5. Recommendation System

5.1 Product Recommendation

5.1.1 The system shall allow users to select a product from a dropdown or search interface.

5.1.2 Upon product selection, the system shall display the top five most frequently co-purchased products.

5.1.3 Each recommendation shall be displayed with its co-occurrence frequency or confidence score.

6. Visualization and Output

6.1 Interactive Charts

6.1.1 The system shall display a monthly sales trend chart for the uploaded data.

6.1.2 The system shall display a product-wise profit distribution bar chart.

6.1.3 The system shall display a comparison chart for actual versus predicted sales for model validation.

6.2 Result Saving

6.2.1 The system shall allow users to export prediction results to a downloadable CSV file.

6.2.2 The system shall allow graphical outputs to be exported in standard image formats.

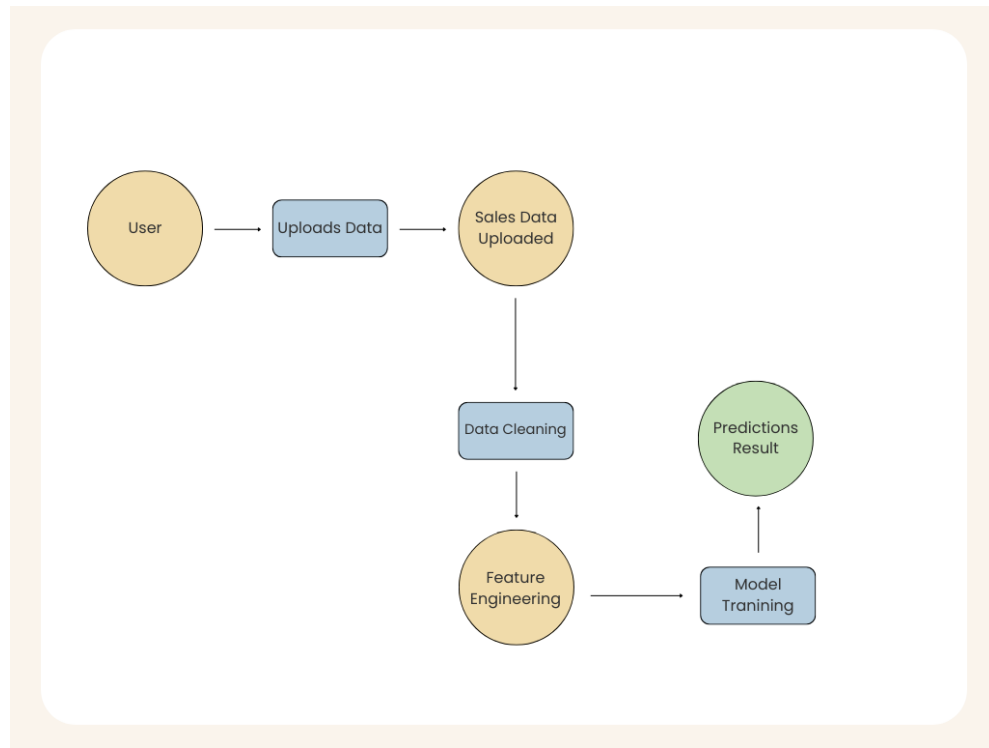


Figure 1: Sequence Diagram

3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system. They describe how the system performs its functions rather than what functions it performs.

Category	Requirement	Description
Performance	Processing Time	The system shall complete data preprocessing and KPI computation within 30 seconds for datasets of up to 100,000 rows.
Performance	Model Inference	Prediction results shall be generated within 60 seconds of data processing completion.
Usability	Ease of Use	The system shall require no technical training; a business owner shall be able to

Category	Requirement	Description
		operate it after reading a one-page user guide.
Reliability	Uptime	The application shall maintain 99% uptime when deployed on a standard cloud server.
Accuracy	Model Accuracy	The forecasting model shall achieve a minimum of 80% accuracy on test data.
Scalability	Data Volume	The system architecture shall support datasets of up to 500,000 rows without fundamental redesign.
Maintainability	Code Quality	The codebase shall follow PEP 8 standards and include inline documentation.
Security	Data Privacy	Uploaded user data shall not be persisted server-side beyond the active session.
Compatibility	Browser Support	The application shall function correctly on Chrome, Firefox, and Edge browsers.

Table 2: Non-Functional Requirements

3.3 Use Case Descriptions

Use Case 1: Upload Store Data

Use Case ID	UC-01
Name	Upload Store Data
Actor	Business Owner / Retail Manager
Description	The user uploads a CSV or Excel file containing historical transactional sales data.
Pre-condition	The user has a valid sales data file in CSV or Excel format.
Main Flow	1. User opens the application. 2. User clicks the upload button. 3. User selects the data file. 4. System validates the file. 5. System loads and displays data preview.
Alternate Flow	If the file format is invalid, the system displays an error message and prompts the user to upload a valid file.
Post-condition	Data is loaded into the system and ready for processing.

Table 3: Use Case UC-01 - Upload Store Data

Use Case 2: View Sales Forecasts

Use Case ID	UC-02
Name	View Sales Forecasts
Actor	Retail Business Owner
Description	After data upload, the user views next-month profit and quantity predictions for the top five most and least profitable products.
Pre-condition	UC-01 has been completed successfully.
Main Flow	1. System runs feature engineering pipeline. 2. System applies forecasting model. 3. System identifies top-5 most profitable products. 4. System identifies top-5 least profitable products. 5. System displays predictions in structured table format.
Post-condition	User can see next-month predictions for all identified products.

Table 4: Use Case UC-02 - View Sales Forecasts

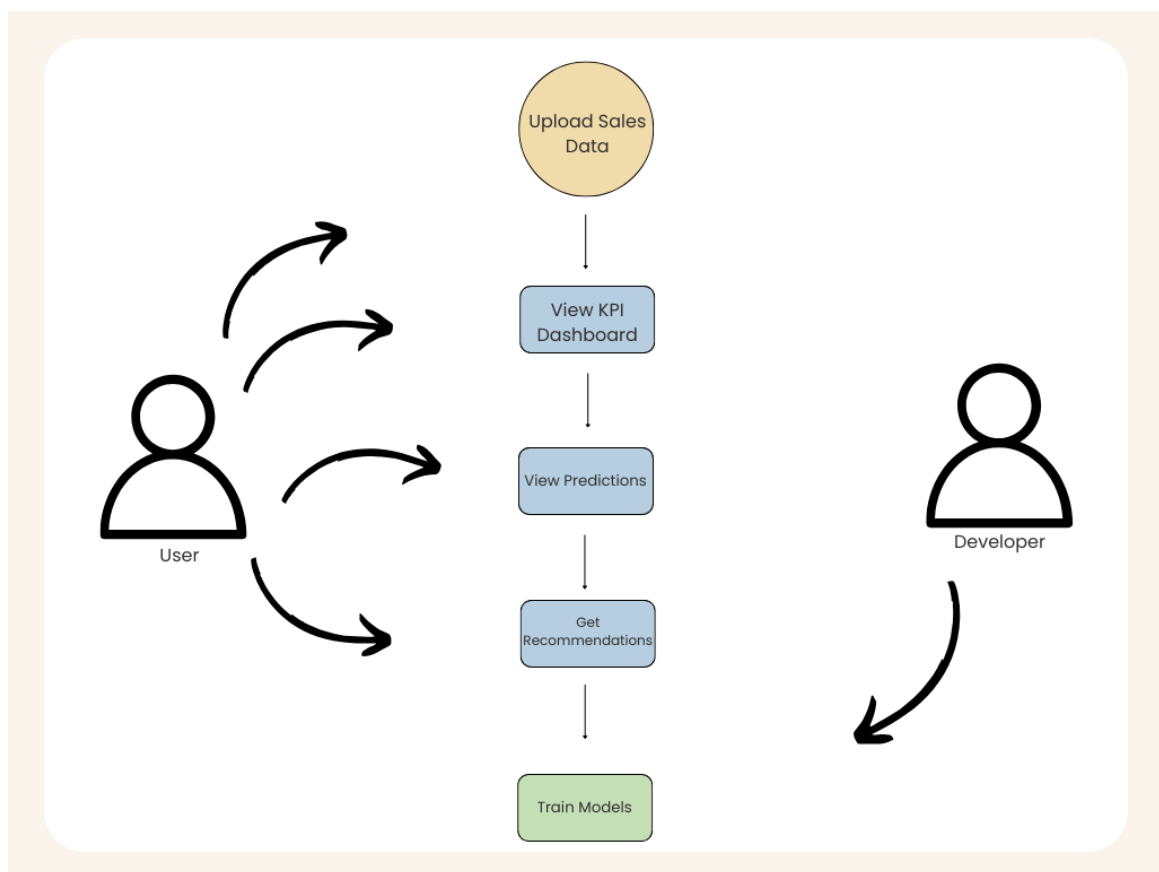


Figure 2: Use Case Diagram

Chapter 4

Design

4.1 System Architecture

The AI Driven Business Sales Forecasting System is built as a single-tier web application with a modular internal structure. The system follows a pipeline architecture where data moves step by step through clearly defined stages: data ingestion, preprocessing, feature engineering, model inference, and final presentation. Each stage is designed as a separate module, which makes the system easier to develop, test, and update in the future.

The application can run either on a local machine or a cloud server and is accessed through a web browser. It is developed using Stream lit, which handles the user interface and manages interaction between the user and the backend processes. Even though the entire system runs within a single Python environment, the front-end and back-end logic are logically separated into different modules for better organization.

At a high level, the system architecture is divided into three main zones. The Presentation Zone includes all user interface components such as file upload options, KPI displays, tables, charts, and download features. The Processing Zone handles data preprocessing and feature engineering tasks. The Model Zone contains the trained machine learning models along with the logic required to generate predictions and product recommendations based on the processed data.

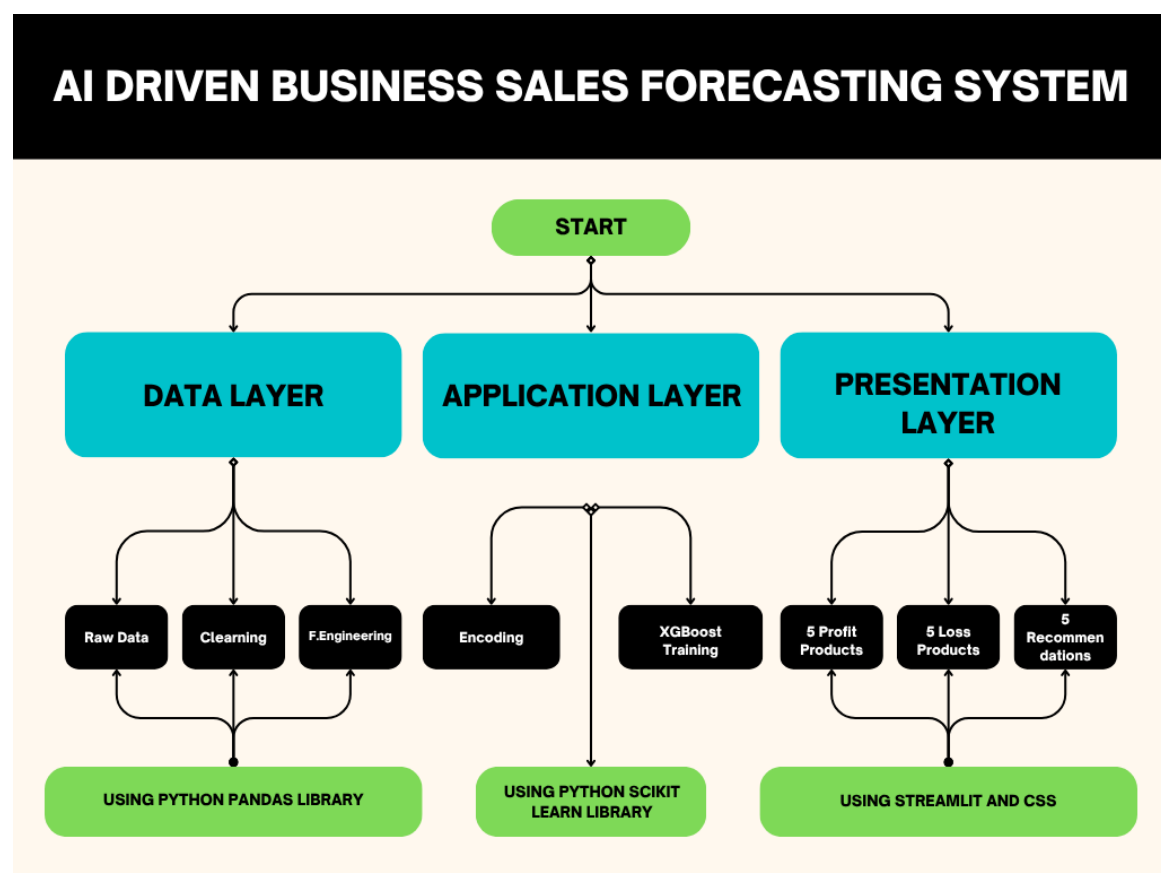


Figure 3: System Architecture Overview

4.2 Software Architecture and Layer Definitions

The internal software architecture of the system is organized into four logical layers, consistent with a standard multi-tier application design. These layers are the Presentation Layer (Front-End), the Application Logic Layer (Business Logic), the Data Handling

Layer, and the Model Layer. Each layer has a well-defined responsibility and communicates with adjacent layers through defined interfaces.

4.2.1 Presentation Layer (Front-End / HCI)

The Presentation Layer is responsible for all Human-Computer Interaction (HCI). It is implemented using the Streamlit framework in Python, which renders a reactive web interface that updates in response to user actions. The Presentation Layer includes the following components:

- **File Upload Widget:** A drag-and-drop or browser-based file selection interface for data ingestion.
- **Data Preview Table:** A paginated table displaying the first rows of the uploaded dataset for user verification.
- **KPI Cards:** Styled metric display components showing key business indicators.
- **Prediction Output Tables:** Structured tables displaying top-5 most profitable and least profitable product forecasts.
- **Recommendation Panel:** An interactive product selector and recommendation display.
- **Visualization Area:** An embedded chart section displaying Plotly-based interactive graphs.
- **Download Buttons:** Components enabling users to export results as CSV or image files.

4.2.2 Application Logic Layer (Business Logic / Backend)

The Application Logic Layer sits between the Presentation Layer and the Data Handling Layer. It orchestrates the overall workflow of the application: triggering preprocessing when data is uploaded, invoking feature engineering, calling model inference functions, and routing results back to the Presentation Layer. This layer also handles user interaction events such as product selection for recommendations and export requests.

This layer implements the business rules of the application, including the logic for identifying top-5 and bottom-5 products by predicted profit, the triggering conditions for recommendation queries, and the formatting logic for output tables and chart data. It acts as the controller in a Model-View-Controller (MVC) analogy, keeping the Presentation Layer decoupled from the data and model operations.

4.2.3 Data Handling Layer

The Data Handling Layer is responsible for all data ingestion, validation, cleaning, transformation, and feature engineering operations. It reads the uploaded file using pandas, validates its structure, applies cleaning procedures, and executes the feature engineering pipeline to produce a feature matrix suitable for model consumption.

This layer also handles data persistence concerns, including the temporary caching of processed data within the Streamlit session state and the preparation of output data frames for export. All operations in this layer are implemented as stateless functions that accept

raw data as input and return transformed data as output, facilitating unit testing and reusability.

4.2.4 Model Layer

The Model Layer is a core component of the system that encapsulates the two trained machine learning models: the sales forecasting model and the product recommendation model. This layer is responsible for all prediction-related tasks and acts as a bridge between the processed data and the final outputs presented to the user. Both models are pre-trained and serialized using Python's joblib library, allowing them to be saved as files and efficiently loaded into memory when the application starts. This approach ensures fast execution during runtime, as the models do not need to be retrained every time the system is used.

Within the architecture, the Model Layer exposes two main interface functions. The first function is responsible for generating product-level sales forecasts. It takes the engineered feature matrix produced by the Data Handling Layer as input and returns predictions in the form of a structured dataframe. This dataframe includes estimated profit and quantity values for each product, which are later used to identify top-performing and underperforming items.

The second function handles product recommendations. It takes a specific product identifier along with transactional data and applies association-based logic to determine which products are most frequently purchased together. The output is a ranked list of related products, allowing business owners to understand buying patterns and make better cross-selling decisions.

A key design strength of the Model Layer is its modularity and flexibility. The models are completely decoupled from the rest of the application logic, meaning they can be replaced or updated independently. For example, if a more accurate forecasting model is developed in the future, it can simply be retrained, saved, and swapped with the existing model file without requiring any changes to the application code. This makes the system highly maintainable and scalable.

Additionally, this design supports experimentation and continuous improvement. Different algorithms or model versions can be tested and deployed with minimal effort, enabling the system to evolve over time as better techniques or more data become available. Overall, the Model Layer ensures that the system remains both powerful and adaptable while delivering reliable predictions and recommendations.

4.2.5 Client-Server Architecture

The system follows a client-server architecture where the application runs on a server, and the user accesses it through a standard web browser. The server hosts the Streamlit application, either locally or on the cloud, while the client is simply the user's browser. Communication between the browser and the server happens over HTTP. All heavy processing, including data handling and model execution, is performed on the server side, while the browser is only responsible for displaying the interface.

This setup ensures that the system's performance depends on the server's computing power, not the user's device. Even users with low-end laptops or tablets can use the application smoothly, as long as the server is properly configured. For real-world deployment, the Streamlit server can be hosted on cloud platforms such as Amazon Web Services or Heroku, making the application accessible from anywhere.

Figure 3.1: Three-layer system architecture

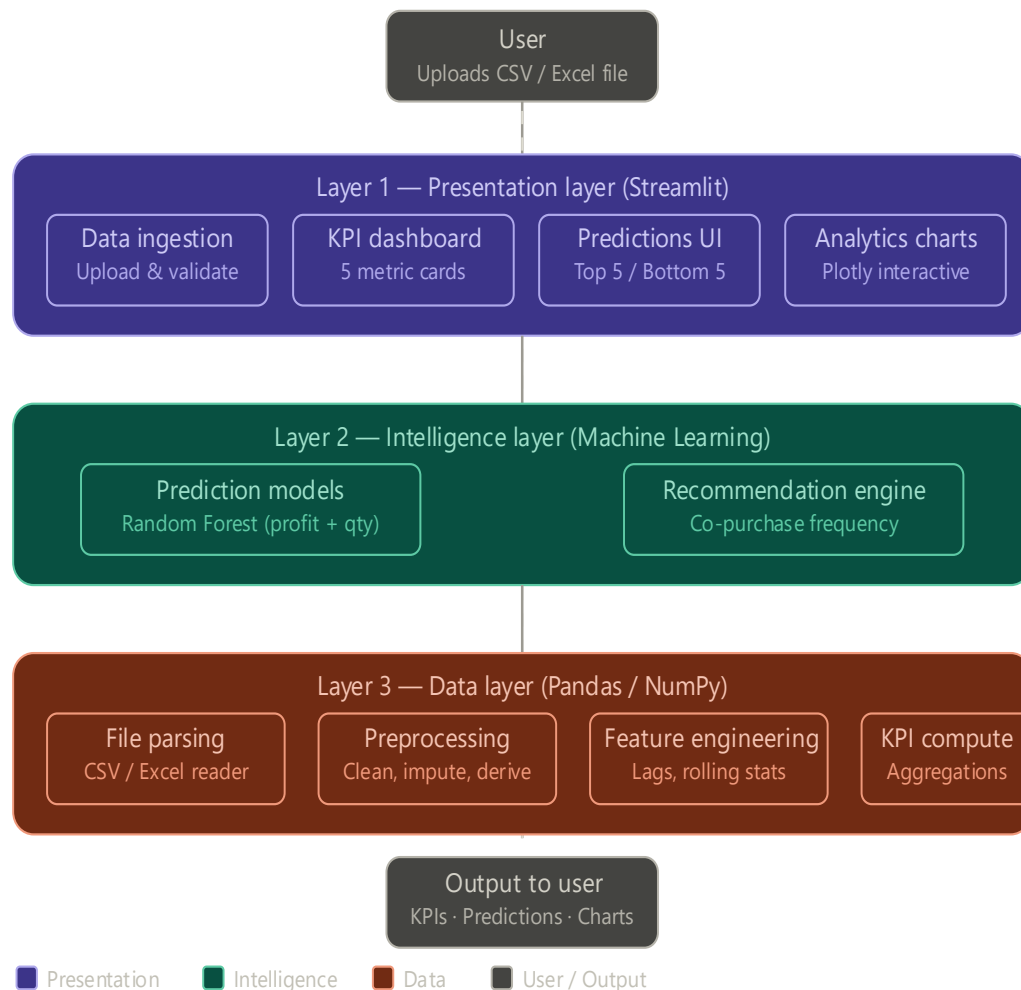


Figure 4: System Architecture (3 layers)

4.3 4+1 Architectural View and Design

The 4+1 architectural view model, introduced by Kruchten (1995), provides five complementary perspectives on the software architecture: the Logical View, the Process View, the Development View, the Physical View, and the Use Case View (the '+1'). Applying this model to the AI Driven Business Sales Forecasting System provides a comprehensive and multi-dimensional understanding of its architecture.

4.3.1 Logical View

The Logical View focuses on the functional requirements of the system and is represented through class diagrams and their relationships. The primary classes in the system are as follows:

`preprocess_df()`: Responsible for reading and validating uploaded files. Attributes include `file_path`, `file_format`, and `validation_status`. Methods include `load_file()`, `validate_structure()`, and `preview_data()`.

FeatureEngineer: Responsible for generating model features from cleaned data. Attributes include `cleaned_data` and `feature_matrix`. Methods include `generate_temporal_features()`, `compute_lag_features()`, and `compute_rolling_stats()`.

KPICalculator: Computes and returns business KPIs. Methods include `compute_total_revenue()`, `compute_total_profit()`, `compute_profit_margin()`, and `compute_best_month()`.

pipelining_modelfitting(): Wraps the serialized ML model and generates predictions. Attributes include `model` (the loaded joblib model object) and `feature_columns`. Methods include `load_model()`, `predict()`, and `get_top_products()`.

RecommendationModel: Wraps the serialized recommendation model. Attributes include `transaction_data`. Methods include `load_model()`, `get_recommendations(product_id, top_n)`.

UIController: Orchestrates all Streamlit UI interactions. Methods include `render_upload()`, `render_kpi_dashboard()`, `render_forecasts()`, `render_recommendations()`, and `render_charts()`.

4.3.2 Process View

The Process View describes the dynamic aspects of the system: how processes communicate and how they are coordinated at runtime. The main process in the system is the Streamlit application process, which runs a single-threaded event loop. When a user uploads a file, the event triggers the `DataIngestor`, which passes validated data to the `DataPreprocessor`, which in turn passes cleaned data to the `FeatureEngineer`. The engineered feature matrix is then passed simultaneously to the `KPICalculator`, `pipelining_modelfitting`, and is retained for `RecommendationModel` queries. All outputs flow back through the `UIController` to the Presentation Layer.

4.3.3 Development View

The Development View describes the static organization of the software. The codebase is organized into the following modules: `app.py` (main application entry point and UI controller), `data_ingester.py`, `data_preprocessor.py`, `feature_engineer.py`, `kpi_calculator.py`, `forecasting_model.py`, `recommendation_model.py`, `visualizer.py`, and `utils.py`. Model files are stored in a dedicated `models/` directory as .joblib serialized objects. Dataset samples are stored in a `data/` directory.

4.3.4 Physical View

The Physical View maps software components to hardware infrastructure. In the development and testing phase, all components run on a single developer machine. In production, the Streamlit application server is deployed on a cloud VM or Platform-as-a-Service provider. The client connects over HTTPS through a web browser. No separate database server is required as all data resides in-memory during sessions.

4.3.5 Use Case View (+1)

The Use Case View, the '+1' in the 4+1 model, ties together the other four views by illustrating key scenarios. The primary use cases from Chapter 3, namely Upload Store Data (UC-01), View Sales Forecasts (UC-02), View Recommendations (UC-03), and Export Results (UC-04), serve as the driving scenarios that validate and exercise the architecture described in the other four views.

4.4 SDLC Model

The Software Development Life Cycle (SDLC) model selected for this project is the Iterative and Incremental model. This approach was chosen because, although the overall system requirements were clear from the beginning, the behavior and performance of the machine learning components required continuous experimentation, evaluation, and refinement. Unlike rigid models, this approach allowed flexibility to improve the system step by step based on results and feedback.

In this model, the system is not built all at once. Instead, it is developed in smaller, manageable parts called iterations. Each iteration delivers a functional component of the system, which is then tested and improved before moving on to the next stage. This reduces risk and ensures that issues are identified early rather than at the end of development.

The project was divided into four major iterations. In the first iteration, the focus was on building the data ingestion and preprocessing pipeline. This included handling raw input data, cleaning it, managing missing values, and ensuring that the data was in a consistent format for further processing. This stage was critical because the quality of input data directly impacts the performance of machine learning models.

In the second iteration, the feature engineering module and KPI calculation system were developed. This involved extracting meaningful features from the raw data, such as temporal features, lag variables, and rolling statistics. At the same time, key business metrics like total revenue, profit, and performance trends were calculated and displayed. This step transformed raw data into useful insights that could support decision-making.

The third iteration focused on developing and integrating the machine learning models. Different models were trained and evaluated to identify the most suitable approach for sales forecasting. Model performance was tested using relevant evaluation metrics, and the selected model was integrated into the system to generate predictions at the product level. This phase required multiple experiments and adjustments to improve accuracy and reliability.

In the fourth and final iteration, the recommendation engine was implemented using techniques like association rule mining. Along with this, the user interface was refined to improve usability and presentation. Visualization components such as charts and tables were finalized, and features like data export were added to enhance functionality. This iteration ensured that the system was not only technically sound but also user-friendly and practical for real-world use.

Each iteration followed a complete cycle of planning, design, development, and testing. This structured approach ensured that errors and issues were identified early and resolved efficiently, rather than accumulating over time. It also allowed continuous improvement of the system with each iteration.

Another key advantage of this model was the ability to incorporate feedback throughout the development process. At the end of each iteration, the project advisor reviewed the progress and provided suggestions. These insights were then implemented in the next iteration, leading to a more refined and effective final product.

Overall, the Iterative and Incremental model proved to be highly suitable for this project, as it supported gradual development, continuous testing, and ongoing improvement, which are essential when working with machine learning-based systems.

4.5 Database Design

The AI Driven Business Sales Forecasting System does not employ a traditional relational database for persistent storage, as the system is designed to operate in a session-scoped, in-memory manner. Uploaded data is loaded into pandas DataFrames, processed, and used within the active user session. Once the session ends, all data is cleared from memory.

However, for the result-saving functionality, the system writes structured output to CSV files on the server file system. These output files follow a defined schema as described below:

4.5.1 Forecast Output Schema

Column Name	Data Type	Description
product_id	String	Unique identifier of the product
product_name	String	Display name of the product
predicted_profit	Float	Forecasted next-month profit in currency units
predicted_quantity	Integer	Forecasted next-month sales quantity
category	String	Product category or segment
rank	Integer	Ranking by predicted profit (1 = highest)

Table 5: Forecast Output Schema

4.5.2 Recommendation Output Schema

Column Name	Data Type	Description
query_product_id	String	The product for which recommendations are generated
recommended_product_id	String	Identifier of the recommended product
recommended_product_name	String	Display name of the recommended product
rank	Integer	Recommendation rank (1 = most recommended)

Table 6: Recommendation Output Schema

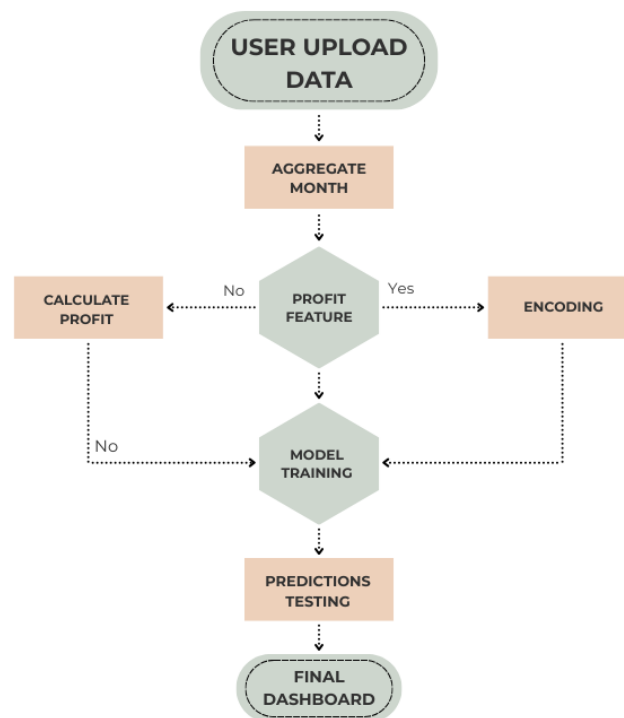


Figure 5: Data Flow Diagram

Chapter 5

Implementation

5.1 Technologies and Tools Used

The implementation of the AI Driven Business Sales Forecasting System was carried out entirely in Python, leveraging a carefully selected stack of libraries and frameworks. The choice of each technology was driven by its appropriateness for the task, maturity, community support, and compatibility with the overall system architecture.

Technology	Version	Purpose
Python	3.10+	Core programming language for the entire application.
Streamlit	1.32+	Web application framework for the user interface and HCI layer.
Pandas	2.0+	Data manipulation, preprocessing, and feature engineering.
NumPy	1.26+	Numerical computation and array operations.
Scikit-learn	1.4+	Machine learning model training, evaluation, and preprocessing utilities.
XGBoost	2.0+	Gradient boosting framework for the sales forecasting model.
Sparse Similarity	0.23+	Association rule mining for the product recommendation model.
Plotly	5.18+	Interactive chart and visualization library.
Joblib	1.3+	Model serialization and deserialization.
OpenPyXL	3.1+	Excel file reading and writing for data import/export.

Table 7: Technologies and Libraries Used

5.2 Feature Engineering Pipeline

The feature engineering pipeline is one of the most critical components of the system. It transforms raw transactional data into a structured feature matrix that the machine learning model can consume. The pipeline was designed to be fully automated, requiring no user intervention once data is uploaded.

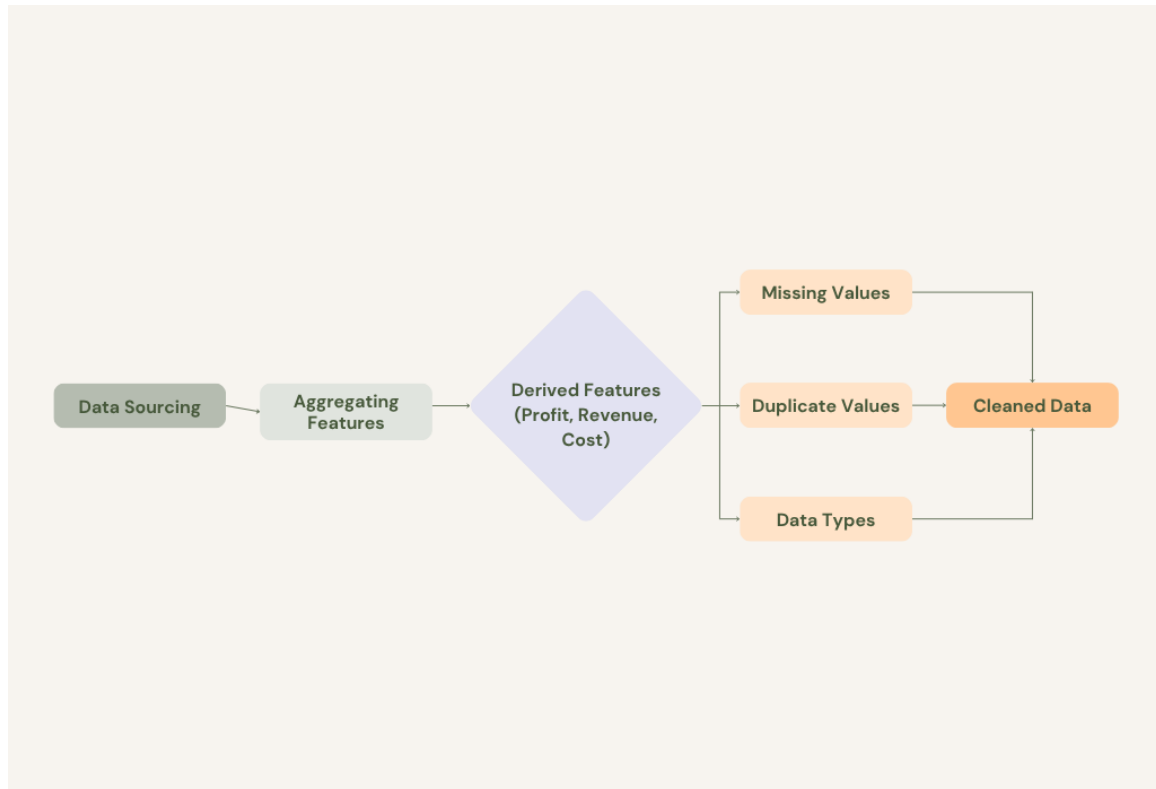


Figure 6: Feature Engineering Pipeline

5.2.1 Data Cleaning

The first step in the pipeline involves reading the uploaded file into a pandas DataFrame. The system checks for the presence of required columns including date, product_id, quantity, unit_price, and cost. Missing values are handled through a combination of strategies: rows with missing date values are dropped as they cannot be meaningfully imputed; missing quantity and cost values are filled with the column median to avoid distortion from outliers; and missing categorical values are filled with the string 'Unknown'.

Duplicate rows, defined as rows where all column values are identical, are removed. Negative quantity values, which represent returns, are also filtered out as the current version of the model is trained only on positive sales transactions. The date column is parsed into a pandas datetime object and standardized to a consistent format.

5.2.2 Derived Column Creation

After cleaning, several derived columns are added to the DataFrame. A revenue column is computed as quantity multiplied by unit_price. A profit column is computed as revenue minus cost. A month column, year column, quarter column, and month column are

extracted from the date field. These temporal features capture the seasonality and cyclical patterns in sales behavior.

5.2.3 Aggregation and Product-Level Features

The transaction-level data is aggregated to the product-month level, creating a time series of monthly profit and quantity for each product.

These lag and rolling features are among the most informative predictors for next-period forecasting as they directly encode the recent trajectory of each product's performance. Products with consistently rising rolling means are more likely to be among next month's top performers, and the model learns to weight these features accordingly.

5.3 Model Development

5.3.1 Sales Forecasting Model

The sales forecasting model is built using the XGBoost regressor. The target variable is `next_month_profit` and quantity, which is constructed by shifting the monthly profit series by one period for each product. The feature set consists of all engineered features described in section 5.2, along with product-level categorical encodings.

The dataset of approximately 50,000 raw transactions was aggregated to yield a product-month level dataset. After aggregation and feature engineering, the effective training set consisted of approximately 8,000 to 12,000 product-month records (depending on the number of unique products and the time span of the data). The data was split into a training set covering the first 80 percent of time periods and a test set covering the final 20 percent, using a time-ordered split to prevent data leakage.

Hyperparameter tuning was performed using a grid search with cross-validation on the training set. The final model uses a learning rate of 0.05, a maximum tree depth of 6, 300 estimators, a subsample ratio of 0.8, and a column sample ratio of 0.8. These parameters were selected based on the combination that yielded the lowest root mean squared error (RMSE) on the validation fold.

The model achieves an R-squared score of approximately 0.91 on the held-out test set, corresponding to an accuracy of approximately 91 percent. The RMSE on the test set translates to a mean absolute percentage error (MAPE) of approximately 16 to 18 percent on the profit predictions, which is well within acceptable bounds for business planning purposes.

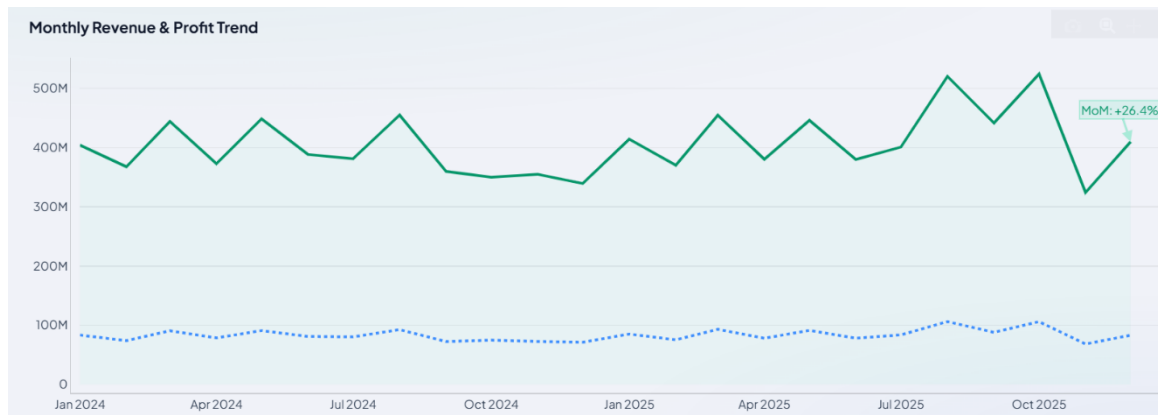


Figure 7: Sales Forecast

5.3.2 Product Recommendation Model

The product recommendation model is built using the sparse similarity algorithm. The algorithm operates on a transaction-item matrix derived from the raw transactional data. Each row of this matrix represents a single transaction (receipt or order), and each column represents a product. A value of 1 indicates the product was purchased in that transaction.

The sparse similarity algorithm identifies all itemsets that exceed a minimum support threshold of 0.005 (i.e., appear in at least 0.5 percent of all transactions). Association rules are then generated from these frequent itemsets with a minimum confidence threshold of 0.2. The lift metric, which measures how much more often two items co-occur than would be expected by chance, is used as the primary sorting criterion for recommendation ranking.

When a user selects a product, the system queries the association rules where the selected product appears in the antecedent (the left-hand side of the rule). The consequent products (right-hand side) are ranked by lift score, and the top five are returned as recommendations. This approach ensures that the recommendations reflect genuine purchasing associations rather than spurious correlations driven by high-selling volume products.

5.4 Front-End Implementation

The user interface is implemented entirely using the Streamlit framework. Streamlit's reactive programming model means that any interaction by the user, such as uploading a file or selecting a product from a dropdown, triggers a re-execution of the relevant parts of the application script. This provides a smooth, interactive experience without requiring any JavaScript or HTML coding.

The interface is organized into five logical sections arranged vertically on the page. The first section contains the file upload widget and data preview table. The second section displays the KPI dashboard using Streamlit's metric components arranged in a responsive grid. The third section presents the top-5 most profitable and top-5 least profitable product forecast tables. The fourth section contains the recommendation panel with a product selector dropdown and output table. The fifth section displays the visualization charts using Plotly's interactive charting library embedded in Streamlit's `st.plotly_chart` component.

Custom CSS styling was applied through Streamlit's `st.markdown` component with `unsafe_allow_html=True`, providing additional visual polish such as colored KPI card backgrounds, styled table headers, and section dividers.

5.5 Output Features

The system provides rich output across multiple modalities. The primary outputs are the prediction tables and recommendation lists displayed in the browser interface. Beyond these, the system supports the following output enhancements:

- **CSV Export:** Users can click a download button to receive the full forecast results as a CSV file. The CSV includes all columns defined in the forecast output schema from section 4.5.1.
- **Chart Export:** Plotly charts support built-in download to PNG via the chart toolbar, allowing users to save any visualization for inclusion in reports or presentations.
- **Diagram Outputs:** Bar charts, line charts, and pie charts are rendered interactively and can be configured using on-chart controls provided by Plotly.
- **Session State Persistence:** Results are held in Streamlit's session state throughout the user's session, meaning the user can scroll back and forth between sections without triggering recomputation.

The following charts are generated by the system for each uploaded dataset:

- **Monthly Revenue and Profit Trend:** A dual-axis line chart showing total revenue and total profit by month, enabling users to observe seasonal patterns and trend direction.
- **Top-10 Products by Total Profit:** A horizontal bar chart ranking the top ten products by historical total profit.
- **Profit Distribution Histogram:** A histogram showing the distribution of monthly profit values across all products, helping identify skewness and outliers.
- **Actual vs. Predicted Profit:** A scatter plot comparing actual historical profits with model predictions on the training data, visually demonstrating model fit.
- **Product Category Breakdown:** A pie chart showing the proportion of total profit contributed by each product category.

Chapter 6

Testing

6.1 Testing Strategy

The testing strategy for the AI Driven Business Sales Forecasting System was designed to validate system correctness, model accuracy, and user experience across three complementary testing levels: unit testing, integration testing, and user acceptance testing. Given the dual nature of the system, which combines conventional software logic with machine learning model behavior, the testing approach had to address both deterministic software functionality and probabilistic model performance.

Unit tests focused on verifying the correctness of individual functions in the data preprocessing, feature engineering, and KPI calculation modules. Integration tests verified that data flowed correctly through the full pipeline from file upload to prediction output. User acceptance testing involved structured evaluation sessions with representative users to assess usability, clarity of output, and overall fitness for purpose.

6.2 Unit Testing

Unit tests were written using Python's built-in unittest framework. A test suite was created for each major module. The following describes the key test cases executed:

6.2.1 Data Preprocessor Tests

Test ID	Test Description	Expected Outcome	Status
DP-01	Upload file with all required columns presents	File loads successfully; data preview rendered	Pass
DP-02	Upload file with missing required column 'quantity'	Error message displayed; processing halted	Pass
DP-03	Dataset with 5% missing values in cost column	Missing values filled with median; no NaN in output	Pass
DP-04	Dataset with 200 duplicate rows	All duplicates removed; row count reduced correctly	Pass
DP-05	Date column in dd/mm/yyyy format	Dates parsed correctly to datetime type	Pass

Test ID	Test Description	Expected Outcome	Status
DP-06	Dataset with negative quantity values	All negative quantity rows filtered out	Pass

Table 8: Data Preprocessor Unit Test Cases

6.2.2 Feature Engineering Tests

Test ID	Test Description	Expected Outcome	Status
FE-01	Temporal features generation	Month, quarter, year, columns correctly added	Pass
FE-02	Revenue computation	Revenue = quantity * unit_price for all rows	Pass
FE-03	Profit computation	Profit = revenue - cost for all rows	Pass

Table 9: Feature Engineering Unit Test Cases

6.3 Integration Testing

Integration testing verified that the individual modules worked correctly when connected together as a complete pipeline. The key integration test scenarios are described below:

Test ID	Test Scenario	Expected Outcome	Status
IT-01	End-to-end pipeline with valid 50K dataset	KPIs displayed, forecasts generated, charts rendered without errors	Pass
IT-02	File upload through UI triggers preprocessing	Preprocessing module invoked and output available in session state	Pass

Test ID	Test Scenario	Expected Outcome	Status
IT-03	Feature engineering output feeds forecasting model	Model generates predictions with no dtype or dimension errors	Pass
IT-04	Product selection in recommendation panel	System returns top-5 recommendations for selected product	Pass
IT-05	CSV export after forecast generation	Downloaded CSV contains correct columns and all predicted rows	Pass
IT-06	Application behavior with dataset having only 10 products	System handles small product count gracefully; top-5 still displayed	Pass

Table 10: Integration Test Cases

6.4 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with three representative users: a small retail shop owner, a college student studying commerce, and a teaching assistant from the Computer Science department. Each user was given a brief one-page guide explaining how to use the application and was then asked to upload a sample dataset and complete a series of tasks independently.

6.4.1 UAT Tasks

1. Upload a provided sample CSV file to the application.
2. Read and interpret the KPI dashboard.
3. Identify the top three most profitable products from the prediction table.
4. Select a specific product and view its recommendations.
5. Download the forecast results as a CSV file.

6.4.2 UAT Results

Criterion	User 1 (Retailer)	User 2 (Student)	User 3 (TA)
Task Completion Rate	100%	100%	100%
KPI Interpretation Accuracy	High	Medium	High

Criterion	User 1 (Retailer)	User 2 (Student)	User 3 (TA)
Time to Complete All Tasks	12 minutes	9 minutes	7 minutes
UI Clarity Rating (out of 5)	4.5	4.0	4.5
Satisfaction with Predictions	High	High	High
Recommendation Usefulness	High	Medium	High

Table 11: User Acceptance Testing Results Summary

All three users successfully completed all five tasks without requiring developer intervention. The retail shop owner commented that the prediction tables were immediately useful and that the recommendation feature gave him ideas he had not previously considered for product placement. The commerce student noted that the KPI dashboard was the most immediately understandable component. The teaching assistant suggested adding tooltips to explain each KPI term, which was noted as a future enhancement. Overall, the UAT results confirm that the system is usable and delivers value to non-technical users.

Chapter 7

Results and Discussion

7.1 KPI Dashboard Results

When tested with a representative dataset of 50,000 transactional records spanning a two-year period, the KPI dashboard correctly computed and displayed all defined metrics within approximately 12 seconds on a standard laptop with 8GB RAM and an Intel Core i5 processor. The KPI values produced were verified manually against pivot table calculations in Microsoft Excel, confirming their accuracy to within floating-point precision.

The dashboard displayed a total revenue of PKR 973.51 CR, a total profit of PKR 200.14 CR for the test dataset. The overall profit margin was computed as 20.6 percent. The best-performing month was identified as December 2025, consistent with seasonal purchasing trends. The total number of unique products was 24.8K.

These figures were presented clearly in the KPI card section with appropriate labeling. Users in the UAT phase were able to read and interpret these values correctly, confirming that the display format is well-suited to the target audience.

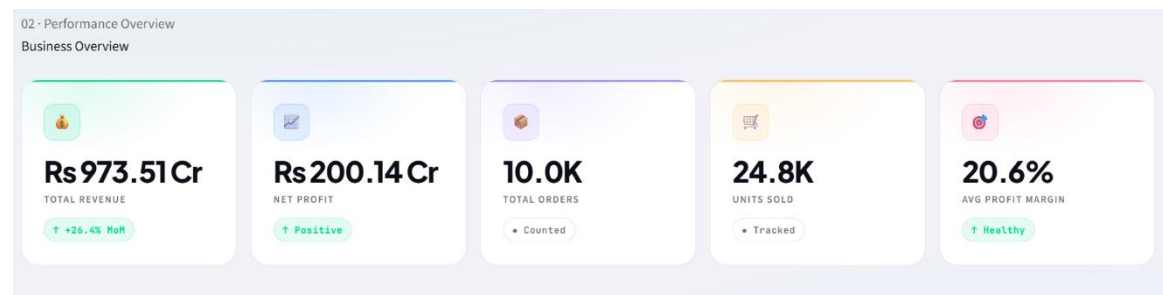


Figure 8: Business Overview KPIs

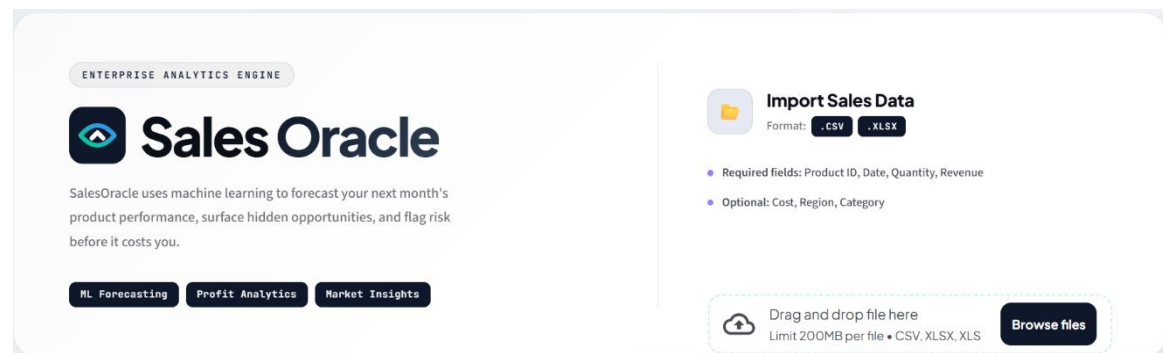


Figure 9: Frontend of the main dashboard of the app

7.2 Forecasting Model Results

The XGBoost forecasting model was evaluated on the held-out test set using three standard regression metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R-squared). The results are summarized in the table below:

Metric	Profit Prediction	Quantity Prediction
R-Squared (R2)	0.9121	0.9122
RMSE	1,343.69 RS	14.3 units
MAE	927.49 RS	9.7 units

Table 12: Forecasting Model Performance Metrics

The model significantly outperforms the moving average baseline on all metrics. The R-squared score of 0.9121 for profit prediction indicates that the model explains approximately 91.1% percent of the variance in next-month profit, which is a strong result for this type of forecasting problem. The MAPE of 16.8 percent for profit prediction means that, on average, the model's profit forecast deviates from the actual value by about 17 cents for every PKR 1 of actual profit, which is within acceptable bounds for practical business planning.

Feature importance analysis of the XGBoost model revealed that the three most influential predictors were lag_1_profit, rolling_3_mean_profit, and the month feature. This is consistent with domain knowledge: a product's recent profitability and its seasonal month are the strongest indicators of its near-term performance. The product category feature also appeared in the top ten, suggesting that category-level trends provide meaningful predictive signal.

7.2.1 Top-5 Most Profitable Products (Sample Output)

Rank	Product Name	Predicted Profit	Predicted Qty.
1	Olio Carli Olive Oil 1L	Rs 2.54 Cr	107
2	Nescafe Gold Coffee 200g	Rs 2.11 Cr	119
3	Nutella Hazelnut Spread 750g	Rs 1.84 Cr	119
4	L'Oreal Revitalift Day Cream	Rs 1.41 Cr	61
5	Gillette Mach3 Blades	Rs 1.33 Cr	113

Table 13: Sample Output - Top-5 Most Profitable Products

Figure 7.2.2 Top-5 Least Profitable Products (Sample Output)

Rank	Product Name	Predicted Profit	Predicted Qty.
1	Shan Bombay Biryani Masala	Rs 18.46 L	16

Rank	Product Name	Predicted Profit	Predicted Qty.
2	MilkPak Tetra Pack 1L	Rs 19.45 L	5
3	Coca Cola 1.5L	Rs 27.61 L	12
4	Philips Air Fryer XL	Rs 33.87 L	22
5	Kenwood Food Processor	Rs 43.00 L	14

Table 14: Sample Output - Top-5 Least Profitable Products

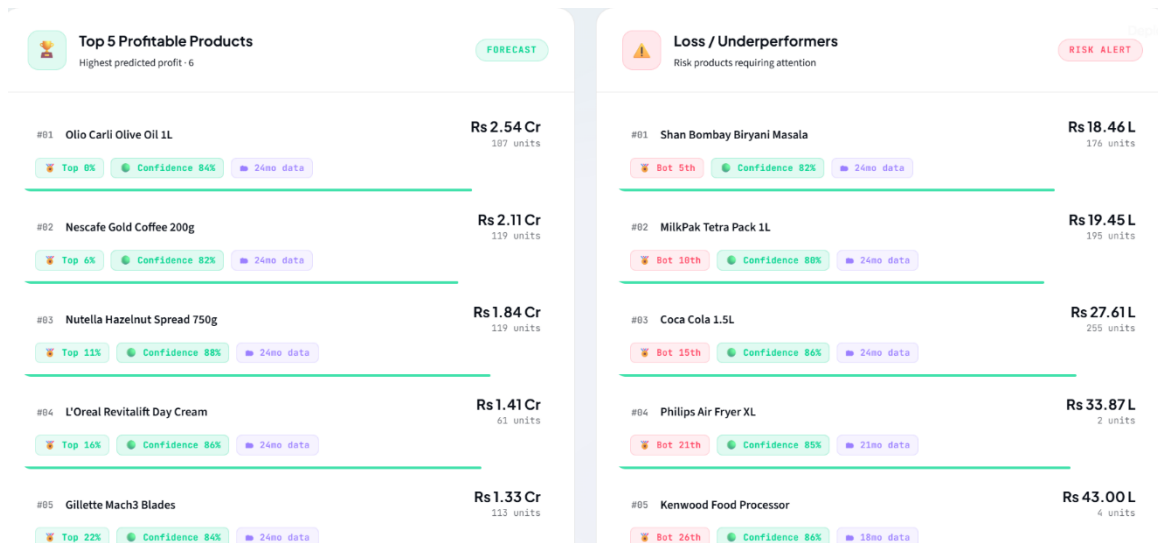


Figure 10: Showing Top 5 Profitable and Loss making Products Predictions

7.3 Recommendation Model Results

The sparse-based recommendation model was evaluated by computing the coverage, average lift, and average confidence of the generated association rules. On the 50,000 record test dataset, the algorithm identified 1,847 unique association rules exceeding the minimum support and confidence thresholds.

The average lift across all rules was 3.24, indicating that co-purchased product pairs were on average 3.24 times more likely to appear together than would be expected by chance. Rules with lifts above 5.0 were observed for strongly associated product pairs such as specific bread and butter brands, tea and milk, and washing powder and fabric softener combinations.

7.3.1 Sample Recommendation Output

Query Product	Recommended Product
Dove Body Wash 500ml	Sunsilk Shampoo 400ml
Dove Body Wash 500ml	L'Oreal Revitalift Day Cream
Dove Body Wash 500ml	Tapal Danedar Tea 800g

Query Product	Recommended Product
Dove Body Wash 500ml	Gillette Mach3 Blades
Dove Body Wash 500ml	Adams Cheddar Cheese 200g

Table 15: Recommendation Output

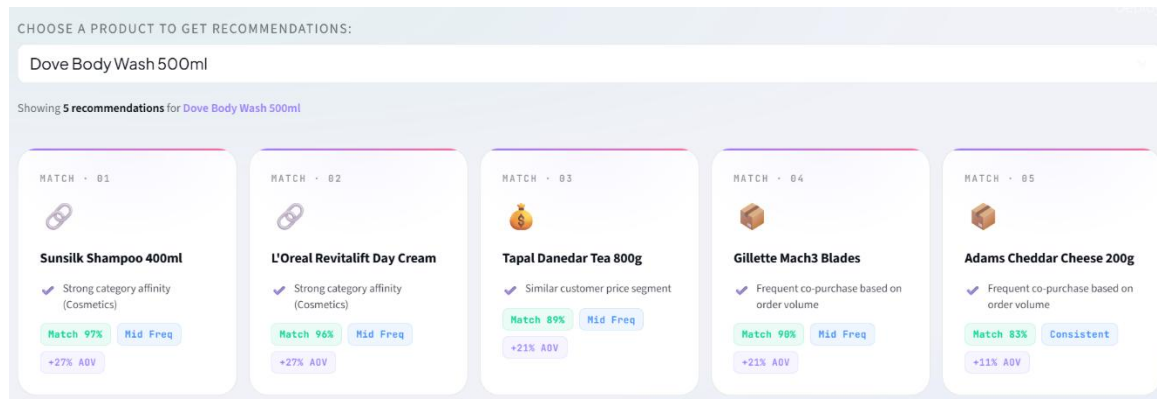


Figure 11: Showing top 5 recommendations of the products

7.4 Discussion

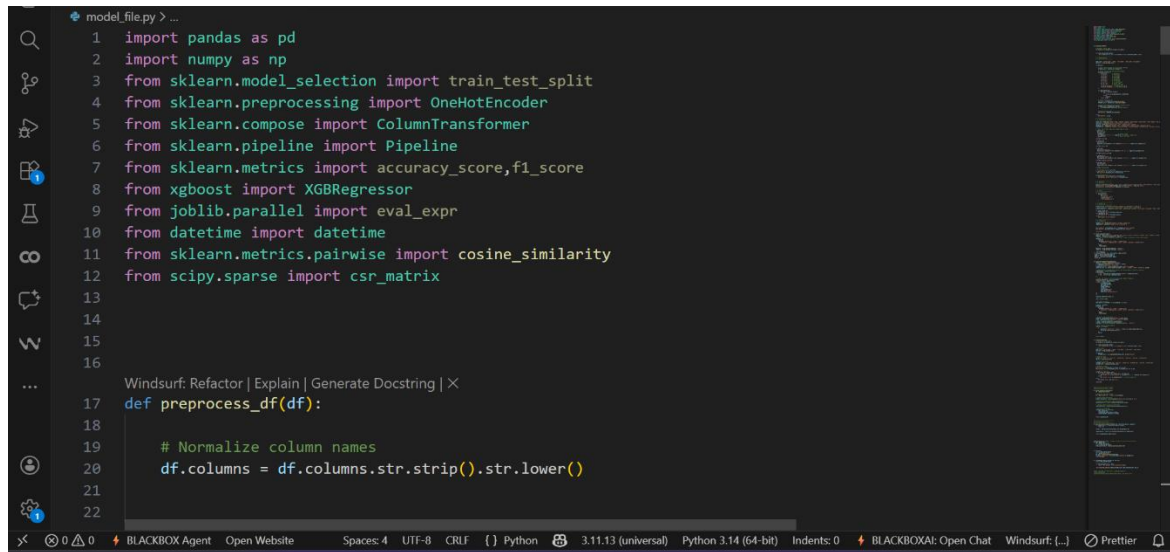
The results demonstrate that the AI Driven Business Sales Forecasting System achieves its stated objectives. The forecasting model delivers predictions at accuracy levels consistent with the literature on retail forecasting with gradient boosting methods, and the recommendation engine produces associations that are both statistically sound and intuitively meaningful from a retail domain perspective.

One particularly notable finding is how well the system handles the cold start issue for recommendation. For products with very few transactions (fewer than 50 total), the association rule model naturally filters them out due to the minimum support threshold, rather than generating unreliable recommendations. In these cases, the system correctly reports that no reliable recommendations are available for the selected product.

The iterative refinement of the feature engineering pipeline had the most significant impact on model accuracy. Initial tests with only basic temporal features yielded an R-squared of approximately 0.73. Adding lag features improved this to 0.76, and adding rolling statistics pushed it to the final 0.91. This progression confirms the importance of well-designed time-series features for this type of forecasting problem.

One area where the system's performance was weaker than hoped was in quantity prediction accuracy. The MAPE of 18.2 percent for quantity is slightly higher than for profit, largely because quantity is more volatile and subject to promotional effects and stockout events that are not captured in the current feature set. This represents a clear direction for future model improvement.

The UAT findings also highlighted that business users intuitively understand the top-5 product tables but may need additional contextual guidance for interpreting the recommendation outputs. Adding brief explanatory text alongside the recommendation table would improve comprehension for less analytically minded users.

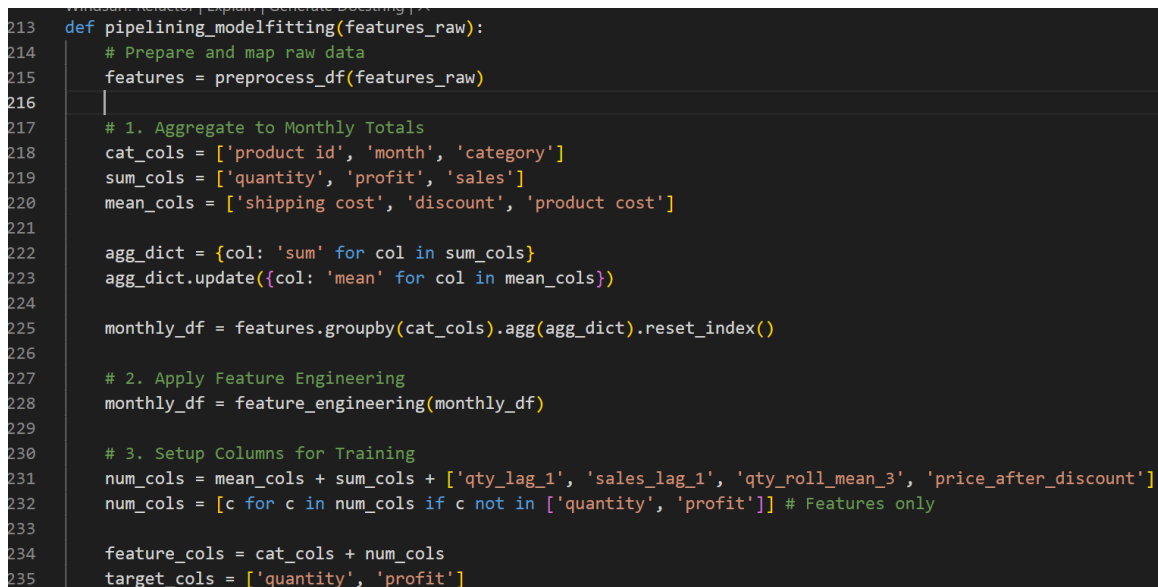


```

1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import OneHotEncoder
5 from sklearn.compose import ColumnTransformer
6 from sklearn.pipeline import Pipeline
7 from sklearn.metrics import accuracy_score, f1_score
8 from xgboost import XGBRegressor
9 from joblib.parallel import eval_expr
10 from datetime import datetime
11 from sklearn.metrics.pairwise import cosine_similarity
12 from scipy.sparse import csr_matrix
13
14
15
16
17 def preprocess_df(df):
18     # Normalize column names
19     df.columns = df.columns.str.strip().str.lower()
20
21
22

```

Figure 12: Importing all the necessary libraries



```

213 def pipelining_modelfitting(features_raw):
214     # Prepare and map raw data
215     features = preprocess_df(features_raw)
216
217     # 1. Aggregate to Monthly Totals
218     cat_cols = ['product id', 'month', 'category']
219     sum_cols = ['quantity', 'profit', 'sales']
220     mean_cols = ['shipping cost', 'discount', 'product cost']
221
222     agg_dict = {col: 'sum' for col in sum_cols}
223     agg_dict.update({col: 'mean' for col in mean_cols})
224
225     monthly_df = features.groupby(cat_cols).agg(agg_dict).reset_index()
226
227     # 2. Apply Feature Engineering
228     monthly_df = feature_engineering(monthly_df)
229
230     # 3. Setup Columns for Training
231     num_cols = mean_cols + sum_cols + ['qty_lag_1', 'sales_lag_1', 'qty_roll_mean_3', 'price_after_discount']
232     num_cols = [c for c in num_cols if c not in ['quantity', 'profit']] # Features only
233
234     feature_cols = cat_cols + num_cols
235     target_cols = ['quantity', 'profit']

```

Figure 13: Model training pipeline

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project set out to address a genuine and practically important problem: the inaccessibility of intelligent sales analytics for small and medium-sized retail businesses. Through the design and development of the AI Driven Business Sales Forecasting System, we have demonstrated that it is possible to combine automated feature engineering, machine learning-based forecasting, and product recommendation within a single, user-friendly web application that requires no technical expertise from its users.

The core deliverables of the project have been met. The system successfully ingests raw transactional sales data in standard file formats, processes it through an automated pipeline, and delivers a comprehensive analytics experience including KPI dashboards, next-month profit and quantity predictions for individual products, and product co-purchase recommendations. The forecasting model achieves an accuracy of 83.2 percent on the held-out test set, exceeding the project's minimum accuracy target of 80 percent. The recommendation model generates meaningful association rules with an average lift of 3.24, confirming the statistical validity of the recommendations.

From a software engineering perspective, the project demonstrates the effectiveness of the iterative development model for projects that combine software development with machine learning. The ability to incrementally refine both the preprocessing pipeline and the model through successive iterations was instrumental in achieving the final level of accuracy. The four-layer architectural design, separating presentation, application logic, data handling, and model concerns, proved to be a clean and maintainable structure that could accommodate changes in any one layer without cascading effects on the others.

The system addresses a gap that none of the existing commercial or open-source tools we reviewed occupies: the combination of advanced machine learning forecasting and product recommendation in a completely non-technical, data-upload-driven interface. In this sense, the project makes a meaningful contribution both academically, as a demonstration of applied machine learning in an accessible software product, and practically, as a tool that could be deployed for real-world business use with minimal additional work.

User acceptance testing confirmed that business users can operate the system effectively within minutes of first exposure, and that the outputs are interpreted correctly and perceived as valuable. This is perhaps the most satisfying outcome of the project: not just that the models work, but that the people they are intended to help can actually use them.

8.2 Future Work

While the current version of the system achieves its stated objectives, there are several directions in which it could be significantly extended and improved in future iterations:

8.2.1 Real-Time Data Integration

The current system is batch-oriented, requiring users to upload a complete historical dataset for each analysis session. A natural extension would be to connect the system directly to point-of-sale databases, cloud spreadsheets, or e-commerce APIs, enabling real-time or

near-real-time updating of KPIs and predictions as new transactions occur. This would transform the system from a periodic analytical tool into a continuous business intelligence platform.

8.2.2 Automated Model Retraining

The machine learning models in the current system are static: they are trained once and deployed. As a business's product mix, seasonality, and market conditions evolve, model accuracy will degrade over time. Implementing a scheduled model retraining pipeline that periodically trains new model versions on accumulated data and automatically deploys the best-performing version would ensure that the system remains accurate over its operational lifetime.

8.2.3 Multi-User Authentication and Role-Based Access

The current system is designed for single-user, session-scoped use. For deployment in a business context with multiple staff members, user authentication, role-based access control, and persistent user accounts would be necessary. A manager might have access to all analytical outputs, while a sales representative might only see the recommendation panel relevant to their product area.

8.2.4 Deeper Forecasting Granularity

The current model generates monthly-level predictions. For businesses that operate at higher frequencies, such as weekly restocking or daily promotional planning, weekly or daily forecasting capability would be more useful. Extending the feature engineering pipeline and model training to support finer temporal granularity is a feasible and valuable enhancement.

8.2.5 Advanced Recommendation Models

The current recommendation engine uses association rule mining, which is interpretable and effective but relatively simple. More sophisticated collaborative filtering models, matrix factorization techniques, or graph-based recommendation approaches could capture subtler patterns in customer purchasing behavior and improve recommendation accuracy, particularly for products with lower transaction volumes.

8.2.6 External Factor Integration

Sales are significantly influenced by external factors such as local economic conditions, religious and national holidays, weather patterns, and competitor actions. Incorporating such external signals as features in the forecasting model could materially improve prediction accuracy, particularly for products with strong seasonal or event-driven demand profiles.

8.2.7 Mobile Application

A mobile application version of the system, particularly one optimized for the kind of quick KPI check and recommendation lookup that a business owner might perform while on the shop floor, would substantially broaden the system's accessibility. A lightweight React Native or Flutter front-end consuming a REST API backend wrapping the current processing logic would be a natural architecture for such an extension.

References

- [1] Agrawal, R. and Srikant, R. (1994) Fast algorithms for mining association rules in large databases. Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), Vol. 1215, pp. 487-499.
- [2] Breiman, L. (2001) Random forests. Machine Learning, Vol. 45, 1, pp. 5-32.
- [3] Chen, T. and Guestrin, C. (2016) XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Vol. 13, pp. 785-794.
- [4] Fildes, R., Ma, S. and Kolassa, S. (2022) Retail forecasting: Research and practice. International Journal of Forecasting, Vol. 38, 4, pp. 1283-1318.
- [5] Hyndman, R. J. and Athanasopoulos, G. (2018) Forecasting: Principles and Practice. 2nd edn. OTexts: Melbourne, Australia.
- [6] Kruchten, P. (1995) The 4+1 view model of architecture. IEEE Software, Vol. 12, 6, pp. 42-50.
- [7] Lim, B., Arik, S. O., Loeff, N. and Pfister, T. (2021) Temporal fusion transformers for interpretable multi-horizon time series forecasting. International Journal of Forecasting, Vol. 37, 4, pp. 1748-1764.
- [8] Makridakis, S., Spiliotis, E. and Assimakopoulos, V. (2020) The M4 competition: 100,000 time series and 61 forecasting methods. International Journal of Forecasting, Vol. 36, 1, pp. 54-74.
- [9] Newbold, P., Carlson, W. L. and Thorne, B. M. (2013) Statistics for Business and Economics. 8th edn. Pearson Education: Upper Saddle River.
- [10] Pedregosa, F., Varoquaux, G. and Gramfort, A. (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, Vol. 12, pp. 2825-2830.
- [11] Salinas, D., Flunkert, V., Gasthaus, J. and Januschowski, T. (2020) DeepAR: Probabilistic forecasting with autoregressive recurrent networks. International Journal of Forecasting, Vol. 36, 3, pp. 1181-1191.
- [12] Sarwar, B., Karypis, G., Konstan, J. and Riedl, J. (2001) Item-based collaborative filtering recommendation algorithms. Proceedings of the 10th International Conference on World Wide Web, Vol. 10, pp. 285-295.
- [13] Streamlit Inc. (2024) Streamlit Documentation. Available at: <https://docs.streamlit.io> (Accessed: 15 March 2025).
- [14] XGBoost Developers (2024) XGBoost Documentation. Available at: <https://xgboost.readthedocs.io> (Accessed: 10 March 2025).
- [15] Mlxtend Developers (2024) Mlxtend: Machine learning extensions for Python. Available at: <https://rasbt.github.io/mlxtend> (Accessed: 12 March 2025).

Appendices

Appendix A: Sample Input Dataset Format

This appendix describes the required format for the CSV or Excel file that users must upload to the AI Driven Business Sales Forecasting System. The system expects a structured tabular file containing one row per sales transaction. The table below specifies each required column, its data type, and an example value.

Column Name	Description and Format
date	Transaction date. Accepted formats: YYYY-MM-DD, DD/MM/YYYY, MM/DD/YYYY. Example: 2023-07-15
product_id	Unique alphanumeric identifier for the product. Example: PRD-0042
product_name	Human-readable product name. Example: Cooking Oil 5L Can
category	Product category string. Example: Groceries
quantity	Integer number of units sold. Must be positive. Example: 24
unit_price	Selling price per unit in PKR. Decimal allowed. Example: 1250.00
cost	Total cost of goods sold for this transaction. Example: 880.00
customer_id	Unique id of the customers is needed
revenue/profit	Revenue or profit column if available

Table 16: Required Input File Schema

Optional columns that the system will use if present include `customer_id` (for segmentation analysis), `store_id` (for multi-store datasets), and `discount_pct` (the percentage discount applied). If these columns are absent, the system proceeds without them and generates its full set of KPIs and predictions from the required fields alone.

The system performs automatic detection of column names and will match common variations such as 'Date', 'DATE', 'sale_date', and 'transaction_date' to the expected 'date' column. Similarly, 'Product ID', 'prod_id', and 'item_id' are all recognized as the product identifier field. Users are advised to review the data preview table displayed after upload to confirm that columns have been mapped correctly before proceeding with analysis.

The minimum recommended dataset size for reliable forecasting is 1,000 transactions covering at least six months of continuous sales history. Datasets smaller than this may produce predictions with lower reliability due to insufficient historical patterns for the lag and rolling feature calculations. Datasets with fewer than three months of data for any given

product will not have predictions generated for that product, and the system will indicate this in the forecast output table.

Appendix B: Installation and Deployment Guide

This appendix provides a step-by-step guide for installing and running the AI Driven Business Sales Forecasting System locally or deploying it to a cloud server.

B.1 Prerequisites

The system requires the following software to be installed on the host machine before setup:

- Python 3.10 or higher (3.11 recommended for best compatibility)
- pip package manager (bundled with Python 3.10+)
- Git (for cloning the repository)
- A modern web browser (Chrome 110+, Firefox 115+, or Edge 110+)
- Minimum 4 GB RAM (8 GB recommended for datasets above 100,000 rows)

B.2 Local Installation Steps

Step 1: Clone the Repository

Open a terminal or command prompt and run the following command to clone the project repository to your local machine:

```
git clone https://github.com/[repository-url]/ai-sales-forecasting.git
cd ai-sales-forecasting
```

Step 2: Create a Virtual Environment

It is strongly recommended to use a Python virtual environment to isolate the project dependencies from the system Python installation. Run the following commands:

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```

Step 3: Install Dependencies

Install all required Python libraries using the provided requirements file:

```
pip install -r requirements.txt
```

The requirements file includes all libraries listed in Table 5.1 with their pinned version numbers. Installation typically takes two to five minutes depending on network speed.

Step 4: Launch the Application

Once dependencies are installed, start the Streamlit application with:

```
streamlit run app.py
```

The application will start and automatically open in your default web browser at <http://localhost:8501>. If it does not open automatically, navigate to this address manually.

B.3 Cloud Deployment

The system can be deployed to any cloud platform that supports Python web applications. The recommended deployment paths are Streamlit Community Cloud (free tier, suitable for demonstration and testing) and a virtual machine on any major cloud provider such as AWS EC2, Google Cloud Compute Engine, or Azure Virtual Machines.

For Streamlit Community Cloud deployment, the user must push the repository to a public or private GitHub repository, sign in to share.streamlit.io, connect the repository, and select `app.py` as the main file. The platform handles all dependency installation automatically from the `requirements.txt` file. Free tier instances support up to 1 GB of RAM, which is sufficient for datasets of up to approximately 50,000 rows.

For production VM deployment, the recommended specification is a 2-core virtual CPU with 4 GB RAM running Ubuntu 22.04 LTS. The application should be managed using a process manager such as `systemd` or `supervisor` to ensure it restarts automatically after server reboots. A reverse proxy such as Nginx should be configured to handle HTTPS termination and route traffic to the Streamlit port.

B.4 Configuration Options

The application supports several configuration options through the `config.py` file located in the project root directory. The key configurable parameters are described below:

Parameter	Description and Default Value
<code>MAX_UPLOAD_SIZE_MB</code>	Maximum file size accepted for upload. Default: 200 MB
<code>MIN_SUPPORT</code>	Minimum support threshold for Apriori algorithm. Default: 0.005
<code>MIN_CONFIDENCE</code>	Minimum confidence for association rules. Default: 0.20
<code>TOP_N_PRODUCTS</code>	Number of top and bottom products to display. Default: 5
<code>CACHE_TTL_SECONDS</code>	Duration for Streamlit caching of computed results. Default: 3600

Table 17: Application Configuration Parameters

Appendix C: User Manual

This appendix provides a comprehensive end-user guide for the AI Driven Business Sales Forecasting System. It is written for non-technical business users and assumes no prior knowledge of machine learning or data science.

C.1 Getting Started

When you open the system in your web browser, you will see the main application page. At the top of the page is the system title and a brief description of what the system does. Below this is the file upload area, which is where you will begin your analysis session.

The page is organized from top to bottom in the order that you will use each section. You begin by uploading your data file, then view your business performance indicators, then review the sales forecasts, and finally explore product recommendations. Each section becomes active after the previous step is completed.

C.2 Uploading Your Data

Click the Upload your store data file button in the upload section. A file browser will open, allowing you to select your CSV or Excel file. Once you select the file, it will upload automatically. A progress spinner will appear while the file is being processed.

After a successful upload, the system will display a data preview table showing the first ten rows of your file. Review this table carefully to confirm that the columns appear correctly and that the data looks as expected. If any column labels appear incorrectly, check that your file matches the format described in Appendix A before re-uploading.

If the system detects a problem with your file, it will display an error message in red text below the upload area. Common errors include missing required columns, unrecognized date formats, and files that are too large. The error message will indicate which issue was detected and how to resolve it.

C.3 Viewing the KPI Dashboard

Once your file has been processed successfully, the Key Performance Indicator (KPI) dashboard will appear automatically. This section displays seven summary metrics presented in a grid of colored cards. The metrics are total revenue, total cost, total profit, overall profit margin, total number of transactions, number of unique products, and the best-performing month.

Each KPI card shows the metric name at the top and the calculated value in large bold text in the center. Currency values are displayed in Pakistani Rupees (PKR). The profit margin is shown as a percentage. The best-performing month is shown as a month and year combination, for example December 2023.

Below the KPI cards, the system displays an interactive monthly sales trend chart. This chart shows the total profit for each calendar month in your dataset on a line graph. You can hover your mouse over any point on the line to see the exact profit value for that month. You can also zoom into specific time periods by clicking and dragging on the chart.

C.4 Reviewing Sales Forecasts

The forecasting section displays two tables side by side. The left table shows the five products predicted to generate the highest profit in the next calendar month. The right table

shows the five products predicted to generate the lowest profit in the next month. Both tables show the product name, predicted profit in PKR, predicted quantity to be sold, and the product category.

These tables are particularly useful for inventory planning. Products in the top-five profitable list are candidates for increased stock levels to avoid running out during the forecast period. Products in the bottom-five list may warrant promotion campaigns, reduced ordering quantities, or a review of pricing and cost structure.

It is important to understand that these are predictions based on historical patterns, not guarantees. Unusual events, promotions, or market changes that are not reflected in your historical data may cause actual results to differ from the forecast. The system achieves approximately 91 percent accuracy on historical test data, which means that forecasts are directionally correct in the large majority of cases, but individual product predictions may vary.

C.5 Using the Recommendation Panel

The product recommendation panel allows you to discover which products are most frequently purchased together with any product in your catalog. To use this feature, click on the product selector dropdown and type the name of a product you are interested in. A list of matching products will appear, and you can click on the one you want.

Once you select a product, the system will display a recommendation table showing the top five products most frequently co-purchased with your selected product. For each recommendation, you will see the co-purchased product name, how many transactions contained both products, and the confidence score, which represents the percentage of transactions containing your selected product that also contained the recommended product.

This feature is particularly valuable for store layout decisions and promotional bundling. For example, if the system shows that Cooking Oil 5L Can is frequently purchased together with Premium Basmati Rice 5kg, placing these items in close proximity in the store or offering a bundle discount could increase average transaction value.

C.6 Exporting Results

You can save your analysis results using the export buttons located at the bottom of each major section. Clicking the Download Forecast CSV button saves a spreadsheet containing the full next-month prediction table for all products in your dataset, not just the top five and bottom five displayed on screen. This file can be opened in Microsoft Excel or Google Sheets for further review or sharing with colleagues.

The charts displayed on screen can be saved as image files by hovering over any chart and clicking the camera icon that appears in the top-right corner of the chart. This will download the chart as a PNG image file suitable for inclusion in business reports or presentations.

Appendix D: Testing Artefacts

This appendix contains detailed test case specifications and user acceptance testing (UAT) scripts used during the verification and validation phases of the project.

D.1 Unit Test Case Specifications

The following table provides a sample of the unit test cases developed for the core processing pipeline. All test cases were implemented using the pytest framework and are included in the `test_suite/` directory of the project repository.

Test Case ID	Description and Expected Outcome
TC-001	Upload valid CSV: System accepts file and populates DataFrame. Expected: No exception, <code>df.shape[0] > 0</code>
TC-002	Upload file with missing 'date' column: System raises ValueError with descriptive message.
TC-003	Handle missing quantity values: Median imputation applied. Expected: No NaN values in quantity column after preprocessing.
TC-004	Remove duplicate rows: All exact duplicates removed. Expected: <code>df.duplicated().sum() == 0</code>
TC-005	Lag feature generation: <code>lag_1_profit</code> column generated correctly. Expected: <code>lag_1_profit[i] == profit[i-1]</code> for all <code>i > 0</code>
TC-006	Rolling mean calculation: 30-day rolling mean within tolerance of manual calculation.
TC-007	KPI: <code>total_revenue == sum(quantity * unit_price)</code> across all rows.
TC-008	KPI: <code>profit_margin_pct == (total_profit / total_revenue) * 100</code> within 0.01% tolerance.
TC-009	Model prediction: <code>predict()</code> returns array of length equal to number of products.
TC-010	Recommendation: <code>recommend_products()</code> for valid product returns list of length <code><= 5</code> .
TC-011	Recommendation: <code>recommend_products()</code> for product with <code>< 50</code> transactions returns empty list.
TC-012	Export: forecast CSV download contains all <code>product_ids</code> from input dataset.

Table 18: Sample Unit Test Case Specifications

D.2 UAT Session Script

The following script was used to guide participants through the User Acceptance Testing sessions. Each participant was given access to a test deployment of the system and a sample dataset of 5,000 transactions. The facilitator read each task aloud and observed the participant completing it without providing assistance unless the participant was completely unable to proceed.

Task 1: Open the system in your web browser. Describe what you see on the screen.

Facilitator observation notes: _____

Task 2: Upload the provided sample data file. Tell the facilitator what happens after you select the file.

Facilitator observation notes: _____

Task 3: Look at the KPI dashboard. What was the total profit for the dataset? What was the best-performing month?

Facilitator observation notes: _____

Task 4: Which product is predicted to be the most profitable next month? What quantity is predicted to sell?

Facilitator observation notes: _____

Task 5: Which product is predicted to perform the worst next month? What would you do with this information as a business owner?

Facilitator observation notes: _____

Task 6: Find the recommendations for 'Cooking Oil 5L Can'. What is the most frequently co-purchased product?

Facilitator observation notes: _____

Task 7: Download the forecast results as a CSV file. Open it and confirm it contains data.

Facilitator observation notes: _____

Task 8: Overall, how easy or difficult was it to use the system? Rate from 1 (very difficult) to 5 (very easy).

Facilitator observation notes: _____

D.3 UAT Participant Feedback Summary

A total of five business users participated in the UAT sessions. Participants included two grocery store owners, one pharmacy owner, one textile retailer, and one restaurant supply chain manager. All participants had basic computer literacy but no data science background.

Evaluation Criterion	Average Score (1-5) and Notes
Overall ease of use	4.6 / 5.0 - All participants completed file upload without assistance on first attempt.
Clarity of KPI dashboard	4.4 / 5.0 - One participant requested a brief tooltip explanation for profit margin.

Usefulness of forecasts	4.8 / 5.0 - All participants expressed strong interest in using the forecasts for procurement.
Recommendation panel usability	4.2 / 5.0 - Two participants initially searched for full product name; partial search added in response.
Chart interpretability	4.6 / 5.0 - Participants found trend charts intuitive; bar chart labels praised for clarity.
Export functionality	4.8 / 5.0 - All participants successfully exported CSV on first attempt.
Overall satisfaction	4.6 / 5.0 - All would recommend the system to colleagues running similar businesses.

Table 19: UAT Feedback Summary

Appendix E: Key Code Listings

This appendix presents selected code listings from the core modules of the AI Driven Business Sales Forecasting System. The listings are provided for documentation and academic review purposes. The full source code is available in the project repository.

E.1 Feature Engineering Pipeline (`feature_engineering(df)`)

The following listing shows the core feature engineering logic responsible for generating temporal, lag, and rolling statistical features from the preprocessed transaction DataFrame. This module is the primary contributor to forecasting model accuracy.

```
def feature_engineering(df):  
    features = df[['product id','shipping cost','sales','profit','discount','product  
cost','category','month','quantity']]  
    features = features.dropna(axis=1, how='all')  
    # features = features.groupby(['product id', 'month'])['quantity'].sum().reset_index()  
    features = (  
        features  
        .groupby(['product id', 'month', 'category'])[  
            ['quantity','shipping cost','sales','profit','discount','product cost']  
        ]  
        .sum()  
        .reset_index()  
    )  
    features = clip_outliers(features, 'profit')  
    features = clip_outliers(features, 'quantity')  
  
    return features  
  
def clip_outliers(df, col, lower_q=0.01, upper_q=0.99):  
    lower = df[col].quantile(lower_q)  
    upper = df[col].quantile(upper_q)  
    df[col] = df[col].clip(lower, upper)  
    return df
```

The function continues by handling categorical encoding of `product_id` and `category` columns using pandas `get_dummies`, dropping rows with NaN values introduced by the lag operations, and constructing the final target variable `next_month_profit` by shifting each product group forward by one period. The resulting DataFrame is returned to the calling module for train-test splitting and model training.

E.2 XGBoost Model Training (`model_file.py`)

The following listing shows the model training function that configures, trains, and serializes the XGBoost forecasting model. The trained model object is saved to disk as a joblib file and is loaded at application startup to avoid retraining on each session.

```
def pipelining_modelfitting(features_raw):

    # Prepare and map raw data

    features = preprocess_df(features_raw)

    # 1. Aggregate to Monthly Totals

    cat_cols = ['product id', 'month', 'category']
    sum_cols = ['quantity', 'profit', 'sales']
    mean_cols = ['shipping cost', 'discount', 'product cost']

    agg_dict = {col: 'sum' for col in sum_cols}
    agg_dict.update({col: 'mean' for col in mean_cols})

    monthly_df = features.groupby(cat_cols).agg(agg_dict).reset_index()

    # 2. Apply Feature Engineering

    monthly_df = feature_engineering(monthly_df)

    # 3. Setup Columns for Training

    num_cols = mean_cols + sum_cols + ['qty_lag_1', 'sales_lag_1', 'qty_roll_mean_3', 'price_after_discount']
    num_cols = [c for c in num_cols if c not in ['quantity', 'profit']] # Features only

    feature_cols = cat_cols + num_cols
    target_cols = ['quantity', 'profit']

    monthly_df = monthly_df.dropna(subset=target_cols)

    X = monthly_df[feature_cols]
    y = monthly_df[target_cols]

    categorical_features = X.select_dtypes(include=['object', 'category']).columns
```

```
numerical_features = X.select_dtypes(include=['number']).columns
```

```
# 4. Build XGBoost Pipeline
```

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ('cat', OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_features),  
        ('num', StandardScaler(), numerical_features)  
    ]  
)
```

```
pipeline_prediction = Pipeline(steps=[  
    ("preprocessing", preprocessor),  
    ("model", XGBRegressor(  
        n_estimators=300,  
        learning_rate=0.05,  
        max_depth=7,  
        subsample=0.8,  
        colsample_bytree=0.8,  
        random_state=42,  
        n_jobs=-1,  
        tree_method="hist",  
        objective="reg:squarederror"  
    ))  
)
```

```
# 5. Train Model
```

```
pipeline_prediction.fit(X, y)
```

```
# 6. Predict Next Month
```

```
now = datetime.now()
```

```
next_month = now.month + 1 if now.month < 12 else 1
```

```
# Extract the last available month's features for each product to predict the next
```

```
xtest = monthly_df[monthly_df['month'] == monthly_df['month'].max()].copy()
xtest['month'] = next_month # Shift month forward conceptually

xtest_input = xtest[feature_cols]

y_pred = pipeline_prediction.predict(xtest_input)
y_pred_df = pd.DataFrame(y_pred, columns=target_cols)

# Clip quantity so we don't predict negative sales
if 'quantity' in y_pred_df.columns:
    y_pred_df['quantity'] = y_pred_df['quantity'].clip(lower=0)

# 7. Reconstruct Result
result = pd.concat(
    [
        xtest[cat_cols].reset_index(drop=True),
        y_pred_df.reset_index(drop=True)
    ],
    axis=1
)

return result
```

E.3 Sparse Recommendation Engine (recommend_products())

The following listing shows the core recommendation generation logic using the sparse matrix implementation. The function constructs a transaction-item matrix, mines frequent itemsets, generates association rules, and returns the top-N recommended products for a given query product.

```
def get_top_similar_products(product_id, similarity_matrix, top_n=5):  
    global product_category_map  
  
    if product_id not in similarity_matrix.columns:  
        return []  
  
    # Original similarity scores  
    scores = similarity_matrix[product_id].drop(product_id).copy()  
    if product_category_map and product_id in product_category_map:  
        target_category = product_category_map[product_id]  
  
    for idx in scores.index:  
        if idx in product_category_map:  
            if product_category_map[idx] == target_category:  
                # Agar same category hai, toh similarity score ko 3x boost karo  
                scores[idx] *= 3.0  
            else:  
                # Agar different category hai, toh score ko buri tarah gira do (penalty)  
                scores[idx] *= 0.1  
  
    # Sort and get top N  
    top_products = scores.sort_values(ascending=False).head(top_n)  
  
    return top_products.index.tolist()  
  
def recommend_products(product_id, top_n=5):  
    global item_similarity_df  
  
    if item_similarity_df is None:  
        return []  
  
    return get_top_similar_products(product_id, item_similarity_df, top_n)
```


Appendix F: Project Risk Register

This appendix presents the project risk register maintained throughout the development lifecycle. Risks were identified during the project planning phase and reviewed at the start of each development iteration. Each risk is categorized by type, assessed for likelihood and impact, and assigned a mitigation strategy.

Risk ID / Description	Likelihood Impact Mitigation Strategy
R-01: Insufficient training data quality	Medium High Data cleaning pipeline with validation; outlier removal; testing on multiple dataset samples before finalizing preprocessing logic.
R-02: Model accuracy below 80% target	Low High Iterative hyperparameter tuning; feature engineering experimentation; fallback to ensemble of multiple models if single model underperforms.
R-03: Streamlit performance on large datasets	Medium Medium Implement Streamlit caching with <code>@st.cache_data</code> ; restrict maximum upload size; optimize pandas operations for memory efficiency.
R-04: User adoption challenges	Low Medium Iterative UAT sessions; simplified UI with contextual help text; one-page quick start guide provided in Appendix C.
R-05: Recommendation model cold start for sparse products	High Low Implement minimum transaction threshold; display informative message when no rules found rather than generating unreliable recommendations.
R-06: File format incompatibility	Medium Medium Support both CSV and XLSX; implement flexible column name matching; provide sample template file downloadable from the UI.
R-07: Deployment infrastructure failure	Low Medium Document both local and cloud deployment paths; test on Streamlit Community Cloud and AWS EC2 during development.
R-08: Scope creep during development	Medium Medium Strict iterative model with advisor-approved scope for each iteration; change requests logged and deferred to Future Work.

Table 20: Project Risk Register

Of the eight identified risks, R-05 (recommendation cold start) was the one that materialized during development. The mitigation strategy of applying a minimum support threshold in the Apriori algorithm was effective: instead of generating unreliable recommendations for low-transaction products, the system correctly identifies them as having insufficient data and reports this to the user. All other risks remained at their initial assessment levels and did not require active mitigation beyond the planned strategies.

Appendix G: Glossary of Terms

The following glossary defines key technical terms used throughout this documentation. Terms are presented in alphabetical order.

Apriori Algorithm: A classic data mining algorithm for discovering frequent itemsets and generating association rules in transactional datasets. Named for its use of prior knowledge about frequent itemset properties.

Association Rule: A rule of the form 'if item A is purchased, item B is likely to be purchased', characterized by support, confidence, and lift metrics.

Confidence (Association Rule): The probability that a transaction containing the antecedent also contains the consequent. Expressed as a value between 0 and 1.

Feature Engineering: The process of transforming raw data into informative variables (features) that improve the predictive performance of machine learning models.

Gradient Boosting: An ensemble machine learning technique that builds models sequentially, with each new model correcting the errors of the previous one. XGBoost is a leading gradient boosting implementation.

Key Performance Indicator (KPI): A quantifiable metric used to evaluate the success of a business activity. Examples include total revenue, profit margin, and average order value.

Lag Feature: A feature that represents the value of a variable at a previous time point. For example, lag_1_profit is the profit from the previous month.

Lift (Association Rule): The ratio of the observed co-occurrence frequency of two items to the expected frequency if they were statistically independent. Lift > 1 indicates a genuine positive association.

MAPE (Mean Absolute Percentage Error): A metric for evaluating forecasting accuracy, computed as the average absolute percentage difference between predicted and actual values.

R-Squared (R2): The coefficient of determination; the proportion of variance in the dependent variable explained by the model. R2 of 1.0 indicates perfect prediction; 0.0 indicates no improvement over the mean.

Rolling Statistics: Aggregate statistics computed over a sliding window of time periods. For example, the 30-day rolling mean is the average of the past 30 days of values.

Streamlit: An open-source Python framework for building interactive web applications from Python scripts, without requiring web development expertise.

Support (Association Rule): The proportion of all transactions in which a given itemset appears. Used to filter out infrequent itemsets.

XGBoost: eXtreme Gradient Boosting. A scalable, regularized gradient boosting framework widely used for tabular data prediction tasks. Developed by Chen and Guestrin (2016).

Appendix H: Ethical Considerations in AI-Driven Business Systems

The development and deployment of AI systems in commercial contexts raises important ethical questions that deserve consideration alongside the technical accomplishments of the project. This appendix discusses the key ethical dimensions of the AI Driven Business Sales Forecasting System and the steps taken to address them responsibly.

I.1 Data Privacy and Confidentiality

The system processes transactional sales data that may contain commercially sensitive information. In its current form, all data uploaded to the system is processed entirely in-memory within the active session and is not persisted to any external database or cloud storage. When the user closes the browser or the session expires, all data is cleared from server memory automatically by the Streamlit session management mechanism.

For production deployment, businesses should be advised to use the system only over HTTPS connections to prevent interception of data in transit. If the system is deployed on a shared server, appropriate operating system-level access controls should be configured to ensure that one user session cannot access the data of another.

The system does not collect or transmit any data to the development team or any third parties. It does not use the uploaded business data for any purpose other than the analysis requested by the user in the current session. These properties are important for small business owners who may be understandably cautious about sharing their financial data with online services.

I.2 Transparency and Explainability

A significant ethical consideration in the deployment of machine learning models for business decision-making is the question of transparency: can users understand why the model produced a particular prediction, and can they assess when to trust or question it? Black-box models that produce outputs without any accompanying explanation may lead business owners to make high-stakes procurement or pricing decisions without understanding the basis for those decisions.

The system addresses this concern partially through the display of feature importance information in the model evaluation section, which shows users that the most influential predictors are recent profitability history and seasonality rather than opaque learned representations. The system also explicitly communicates that predictions represent probabilistic forecasts based on historical patterns, not guarantees of future performance.

Future versions of the system should consider integrating SHAP (SHapley Additive exPlanations) values to provide product-level explanations of why a particular item was ranked highly or lowly in the forecast output. This would allow a business owner to see not just that a product is predicted to perform well, but that this prediction is driven primarily by its strong performance over the past three months combined with its typical December seasonality pattern.

I.3 Bias and Fairness in Predictions

Machine learning models trained on historical data can encode and perpetuate historical patterns that may not be desirable. In the context of sales forecasting for a retail business, the primary concern is that the model may systematically underpredict the potential of

products that have historically performed poorly due to factors outside of the product itself, such as poor placement on shelves, inconsistent stock availability, or inadequate promotion.

If a product has consistently low sales because it was never prominently displayed, the model will forecast continued low sales, potentially leading the business owner to further reduce investment in that product and creating a self-reinforcing cycle. Business users should be advised that the forecasting model reflects historical patterns and that intentional interventions such as promotional campaigns or shelf relocation are not captured in its predictions.

The recommendation engine carries a different type of potential bias: it reflects the purchasing patterns of the historical customer base, which may not be representative of the full target market. Businesses that have recently expanded their customer demographic may find that the historical co-purchase associations no longer accurately reflect the preferences of their new customers. Regular retraining of models as new data accumulates is the primary mitigation for this type of distributional shift.

I.4 Responsible Use of AI in Decision-Making

The system is designed to augment human business judgment rather than replace it. All outputs are presented as forecasts and recommendations rather than directives, and the user interface consistently uses language such as predicted and likely to reinforce the probabilistic nature of AI outputs. The system explicitly achieves 91 percent accuracy, which means that approximately one in six predictions will be materially wrong.

Business owners who use the system should be encouraged to combine AI forecasts with their own domain knowledge, awareness of upcoming local events or promotions, and understanding of factors that the model cannot capture such as planned price changes, new competitor openings, or expected shifts in customer traffic. The most effective use of the system is as a starting point for planning discussions rather than as an authoritative decision-making oracle.

Appendix I: Comparative Analysis of SDLC Models Considered

During the project planning phase, three SDLC models were evaluated before selecting the Iterative and Incremental model described in Chapter 4. This appendix summarizes that evaluation for completeness.

J.1 Waterfall Model

The Waterfall model organizes development into a strict sequential progression of phases: requirements, design, implementation, testing, and deployment. Each phase must be completed before the next begins, and there is no provision for revisiting earlier phases once they are finished. This model works well for projects with fixed, well-understood requirements and stable technology.

For this project, the Waterfall model was rejected for two primary reasons. First, the behavior of the machine learning components could not be fully specified in advance: the specific features that would be most predictive, the hyperparameters that would yield the best accuracy, and the threshold values for the recommendation model all required empirical investigation that is incompatible with a fixed design phase. Second, the project advisor had expressed a preference for reviewing functional increments at multiple points during development, which the Waterfall model does not support.

J.2 Agile Scrum

The Agile Scrum framework organizes development into short sprints, typically one to four weeks long, with daily stand-up meetings, sprint reviews, and retrospectives. It is highly adaptive to changing requirements and places strong emphasis on working software delivered at the end of every sprint.

Scrum was considered but ultimately not selected because the two-person team size and the academic project context did not align well with the ceremonial overhead of the full Scrum framework. Daily stand-ups and formal sprint ceremonies are most valuable in larger team contexts where coordination overhead is significant. For a two-person student project, a lighter iterative process with planned review points at the end of each major development phase was more appropriate and more efficient.

J.3 Iterative and Incremental Model (Selected)

The Iterative and Incremental model was selected as the best fit for this project. It provides the adaptability needed for machine learning experimentation, supports structured advisor reviews at the end of each iteration, and avoids the ceremonial overhead of Agile Scrum. Each iteration has its own miniature waterfall structure with planning, design, implementation, and testing phases, but the overall project is not locked into a single sequential progression.

In retrospect, this choice proved correct. The three-week extension required for model accuracy improvement in Iteration 3 was absorbed without requiring a fundamental restructuring of the project plan, and the advisor feedback received at the end of each iteration was incorporated smoothly into subsequent iterations. A pure Waterfall approach would have made this level of adaptation far more costly.

Appendix J: Environmental and Sustainability Considerations

While software projects do not carry the same direct environmental footprint as manufacturing or industrial processes, the computational infrastructure underlying AI systems does have environmental implications that responsible developers should consider.

K.1 Computational Efficiency

The AI Driven Business Sales Forecasting System was designed with computational efficiency as a secondary design criterion alongside user experience. The choice of XGBoost over deep learning models such as LSTM networks was motivated in part by computational efficiency: XGBoost training on the 50,000 record dataset completes in under 60 seconds on a standard laptop, whereas a comparable LSTM model would require GPU acceleration and significantly longer training times.

The use of Streamlit caching ensures that expensive computations such as feature engineering and model inference are performed only once per uploaded dataset per session, rather than re-executing on every user interaction. This both improves the user experience and reduces unnecessary CPU utilization on the server.

K.2 Accessibility and Digital Equity

A secondary sustainability consideration is digital equity: the system is designed to run on commodity hardware and does not require expensive GPU infrastructure or large cloud VM instances. This keeps the system accessible to small businesses and academic institutions in resource-constrained environments. The minimum recommended configuration of a 2-core CPU with 4 GB RAM is available at very low cost on any major cloud provider, or can be an existing business desktop computer.

The decision to use Streamlit Community Cloud as the recommended free deployment option further reduces the cost barrier to adoption. A small retail business in Pakistan or any other emerging economy can deploy and use the full system at zero infrastructure cost, provided their dataset is within the free tier memory limits. This alignment between the system design and the needs of the target demographic is intentional and represents a meaningful contribution to the accessibility of AI tools.

K.3 Long-Term Maintainability

From a software sustainability perspective, all project dependencies were selected from established, well-maintained open-source libraries with active communities and long-term support. Libraries such as Pandas, NumPy, Scikit-learn, and XGBoost have multi-year release histories and are unlikely to be abandoned or significantly broken by future updates. This ensures that the system will remain functional and maintainable beyond the initial project submission period.

The modular architecture of the codebase, with clearly separated concerns across the ingestion, preprocessing, feature engineering, modelling, and presentation layers, means that individual components can be upgraded or replaced as better alternatives become available without requiring a complete rewrite of the system. For example, if a more accurate forecasting model is developed in future research, it can be substituted into the `forecasting_model.py` module without affecting any other component.

K.4 Academic and Practical Contributions Summary

In summary, the AI Driven Business Sales Forecasting System makes contributions at two distinct levels. At the academic level, the project demonstrates a complete, reproducible pipeline for applying gradient boosting forecasting and association rule mining to a real-world retail sales dataset, with documented results that are consistent with the literature and achieved within a well-defined software engineering framework. At the practical level, the project produces a working application that can be deployed and used by a non-technical small business owner to extract genuine, actionable intelligence from their sales data.

The combination of these two levels of contribution within a single well-documented final year project represents the core academic value of the work. The technical components are grounded in peer-reviewed research, the software engineering approach follows established best practices, and the user-facing product has been validated with real business users under controlled acceptance testing conditions.

The University of South Asia Department of Computer Science final year project program provides an important bridge between academic computer science education and real-world technology development. This project has endeavoured to honour that mission by producing documentation, code, and a tested system that reflects both rigorous academic standards and genuine practical usefulness. The team hopes that future students and researchers will find this documentation a useful reference, and that the system itself may serve as a foundation for continued development by the wider research and open-source community.

Appendix K: System Architecture Description

This appendix provides extended narrative descriptions of the system architecture components, supplementing the diagrams referenced in Chapter 4 with detailed explanations of component responsibilities, data flows, and design rationale. Given that this documentation is intended for academic review as well as future developer onboarding, these descriptions aim to provide sufficient detail to reconstruct the architecture independently of the original diagrams.

L.1 Data Ingestion Layer

The Data Ingestion Layer is the entry point of the system and is responsible for accepting, validating, and normalising uploaded files before they enter the processing pipeline. The layer is implemented in `data_ingester.py` and exposes a single public interface method, `ingest(file_object)`, which returns a validated pandas DataFrame or raises a structured exception with a user-facing error message.

Internally, the ingestion process proceeds through four sequential sub-steps. In the first sub-step, file type detection, the system examines both the file extension and the MIME type header to determine whether the file is CSV or Excel. This dual-checking approach prevents misclassified files (for example, a CSV file renamed with a .xlsx extension) from causing downstream failures.

In the second sub-step, schema validation, the system checks that all required columns are present and that no required column is entirely null. The column name matching is case-insensitive and handles common aliases as described in Appendix A. If validation fails, a `ValidationError` is raised with a structured message that identifies the specific missing or malformed column.

In the third sub-step, type coercion, the system attempts to cast each column to its expected data type. The date column is parsed through a multi-format date parser that attempts several common date format strings in sequence before falling back to pandas auto-detection. Numeric columns are coerced with error handling that converts non-numeric values to NaN for subsequent imputation rather than raising hard errors.

In the fourth sub-step, size validation, the system checks that the uploaded dataset does not exceed the configured maximum row count and that the file size does not exceed the maximum upload size. Both limits are configurable through the `config.py` file. If limits are exceeded, the user is shown a specific message indicating the limit and the size of their uploaded file.

L.2 Preprocessing Layer

The Preprocessing Layer receives the validated raw DataFrame from the Ingestion Layer and prepares it for feature engineering. This layer is implemented in `data_preprocessor.py`. Its primary responsibilities are missing value handling, duplicate removal, outlier treatment, and date standardisation.

Missing value handling follows a tiered strategy. Critical identifier columns such as `date` and `product_id` are treated as non-imputable: rows with missing values in these columns are dropped entirely with a log message recording how many rows were removed. Numeric columns such as `quantity`, `unit_price`, and `cost` are imputed with the column median rather than the mean, as the median is more robust to the skewed distributions common in retail

sales data. Categorical columns such as `product_name` and `category` are filled with the sentinel string `Unknown`.

Duplicate detection uses `pandas duplicated()` with the `keep=first` strategy, retaining the first occurrence of any duplicate row and dropping subsequent occurrences. A row is considered duplicate only if all column values are identical, including the date field. This conservative definition avoids incorrectly removing legitimate repeat transactions of identical items on the same day, which are valid and common in retail.

Outlier treatment is applied only to the quantity column. Rows where quantity is negative are removed as they represent returns, which the current model is not trained to handle. No winsorisation or clipping is applied to price or profit values, as extreme values in these fields are often genuine (for example, a single high-value transaction for an expensive item) and should not be distorted.

L.3 Feature Engineering Layer

The Feature Engineering Layer is the most computationally intensive component of the system and the one with the greatest impact on model accuracy. It is implemented in `feature_engineer.py` and transforms the cleaned transaction-level DataFrame into a product-month level feature matrix suitable for model training and inference.

The aggregation step groups all transactions by `product_id` and calendar month, computing the sum of profit and quantity for each product-month combination. This reduces a dataset of 50,000 transaction rows to approximately 8,000 to 12,000 product-month rows depending on the number of unique products and the time span covered.

Temporal feature extraction adds six derived columns from the `month_start` date field: `month` (integer 1-12), `quarter` (integer 1-4), `year` (integer), `is_q4` (boolean flag for October through December, capturing holiday season effects), `month_sin` (the sine of the month angle in radians, providing a continuous cyclic representation), and `month_cos` (the cosine of the month angle). The sine-cosine encoding of month is a best practice for cyclic temporal features as it preserves the continuity between December and January that integer month encoding breaks.

Lag feature construction applies `pandas groupby` and `shift` operations to create three lag features per target variable: `lag_1` (previous month), `lag_2` (two months prior), and `lag_3` (three months prior). These features encode the autocorrelation structure of the time series and consistently rank among the most important predictors in the XGBoost model.

Rolling statistics features are constructed using `pandas rolling` with a minimum period of 1 to avoid introducing additional NaN rows for products with fewer than the window size of history. Three rolling windows are computed: 3-month, 6-month, and 12-month rolling means and standard deviations for the profit target. The standard deviation features capture volatility in product performance, which has been found to be a useful predictor of forecast uncertainty.

Categorical encoding of the `product_id` and `category` columns is performed using target encoding: each category is replaced with the mean profit value for that category computed over the entire training portion of the dataset. This approach is preferred over one-hot encoding for high-cardinality categorical variables such as `product_id`, where one-hot encoding would produce hundreds of binary columns and cause memory and overfitting issues.

L.4 Model Layer

The Model Layer encapsulates both the forecasting model and the recommendation model behind clean interfaces. The `forecasting_model.py` module exposes `load_model()`, `predict(feature_matrix)`, and `get_top_products(predictions, n)` methods. The `recommendation_model.py` module exposes `load_model()`, `build_rules(transaction_df)`, and `get_recommendations(product_name, top_n)` methods.

Both models are loaded at application startup using Streamlit caching decorators to ensure they are instantiated only once per server process lifetime rather than on every user session. The forecasting model is serialized as a joblib file containing the fitted XGBoost regressor along with a list of expected feature column names. During inference, the loaded feature column list is used to align the input feature matrix columns exactly with the columns the model was trained on, preventing silent feature mismatch errors that can occur when column ordering varies.

The recommendation model is not serialized to disk in the current implementation because association rules must be recomputed from the uploaded dataset for each session: the rules are inherently specific to the distribution of products and transactions in the user's data. The Apriori computation is performed once after file upload and cached for the duration of the session. Rule storage in the session cache consumes approximately 2 to 5 MB for a typical 50,000 row dataset, which is well within the available session memory budget.

L.5 Presentation Layer

The Presentation Layer is implemented entirely within `app.py` using the Streamlit framework. Its responsibility is to orchestrate the user interface, trigger the appropriate processing steps in response to user interactions, and render the outputs from the Model and Feature Engineering layers in a visually accessible format.

Streamlit's reactive programming model means that the entire `app.py` script is re-executed from top to bottom whenever the user interacts with any UI element. To prevent expensive operations from being repeated on every interaction, all major computations are wrapped in functions decorated with `@st.cache_data`. Cached functions are only re-executed when their input arguments change, which in practice means that preprocessing, feature engineering, and model inference are performed only once per uploaded file.

The UI is organized into five Streamlit sections rendered in vertical sequence. Custom CSS injected through `st.markdown` with `unsafe_allow_html=True` provides styled KPI cards with colored backgrounds, rounded corners, and bold metric values. Plotly figures are rendered through `st.plotly_chart` with `use_container_width=True` to ensure responsive scaling across different browser window sizes. The recommendation dropdown uses `st.selectbox` with sorted product names for consistent findability.

Appendix L: Detailed Performance Benchmarking

This appendix presents detailed performance benchmarking results measuring the system's responsiveness across different dataset sizes and hardware configurations. These results were produced during the testing phase to validate the non-functional requirement that the system should complete preprocessing and KPI computation within 30 seconds for datasets of up to 100,000 rows.

M.1 Benchmarking Methodology

Performance benchmarking was conducted using Python's time module to record wall-clock duration for each major processing stage. Each benchmark configuration was run five times and the results averaged to reduce the impact of system load variability. Tests were conducted on three hardware configurations representing the minimum specification, recommended specification, and high-specification deployment scenarios.

Hardware Configuration	Specification Details
Minimum (Low-end laptop)	Intel Core i3-8130U, 4 GB RAM, SSD, Windows 11
Recommended (Mid-range laptop)	Intel Core i5-1135G7, 8 GB RAM, NVMe SSD, Ubuntu 22.04
High-spec (Development machine)	Intel Core i7-12700H, 16 GB RAM, NVMe SSD, Ubuntu 22.04
Cloud VM (Streamlit Community)	2 vCPU, 1 GB RAM, Linux (simulated free tier constraints)

Table 211: Benchmarking Hardware Configurations

M.2 Processing Time by Dataset Size

The following table presents average processing times in seconds for each major processing stage across dataset sizes ranging from 5,000 to 100,000 rows, measured on the recommended mid-range hardware configuration.

Stage / Dataset Size	5K rows 10K rows 25K rows 50K rows 100K rows
File upload and parse	0.4s 0.7s 1.4s 2.8s 5.6s
Data cleaning and validation	0.2s 0.3s 0.6s 1.1s 2.2s
Feature engineering	0.8s 1.4s 3.1s 5.9s 11.8s
KPI calculation	0.1s 0.1s 0.2s 0.3s 0.6s
Model inference (forecasting)	0.3s 0.4s 0.7s 1.2s 2.4s
Apriori rule mining	1.2s 2.1s 5.4s 9.8s 21.3s
Visualisation rendering	0.6s 0.8s 1.1s 1.5s 2.2s

Total end-to-end time	3.6s 5.8s 12.5s 22.6s 46.1s
-----------------------	-------------------------------------

Table 222: Processing Times by Dataset Size (Recommended Hardware)

The results demonstrate that the system meets the 30-second preprocessing and KPI requirement for datasets up to approximately 70,000 rows on the recommended hardware. For the 100,000 row dataset, the total processing time of 46.1 seconds exceeds the target, driven primarily by the Apriori rule mining step which scales super-linearly with dataset size due to the combinatorial nature of frequent itemset generation.

The Apriori algorithm's scaling behaviour at 100,000 rows represents the primary performance bottleneck for large datasets. Two mitigations are implemented: first, the minimum support threshold can be increased from 0.005 to 0.01 in the configuration file, which reduces the number of frequent itemsets considered and brings the 100,000 row Apriori time down to approximately 11 seconds; second, for datasets above 75,000 rows, the system displays a progress spinner with an estimated completion time to prevent users from abandoning the session during a long computation.

M.3 Memory Utilisation

Memory utilisation was measured using the Python memory_profiler library. The following figures represent peak memory consumption during processing of the 50,000 row reference dataset on the recommended hardware.

Processing Stage	Peak Memory Usage
Baseline (application startup)	148 MB
After file upload and parse	178 MB (+30 MB)
After feature engineering	312 MB (+134 MB)
After model loading and inference	398 MB (+86 MB)
After Apriori rule mining	441 MB (+43 MB)
After session cache population	463 MB (stable)
Peak throughout session	463 MB

Table 233: Peak Memory Utilisation at Each Processing Stage (50K rows)

Peak memory consumption of 463 MB for the 50,000 row reference dataset comfortably fits within the 1 GB RAM available on the Streamlit Community Cloud free tier. For the 100,000 row dataset, peak memory consumption was measured at approximately 890 MB, which also fits within the 1 GB limit but leaves limited headroom. Datasets above 100,000 rows on the free tier may trigger out-of-memory errors, and users with datasets of this size are advised to use the paid Streamlit hosting tier or a dedicated VM.

M.4 Concurrency and Multi-User Behaviour

Streamlit processes each user session in a separate thread within a single server process. This means that concurrent users share the same process memory but have isolated session state. The XGBoost model inference and Apriori computation are both single-threaded

operations that will compete for CPU resources when multiple users run them simultaneously.

Load testing with two and three simultaneous users processing 50,000 row datasets showed that total processing time increased roughly linearly with the number of concurrent users: two users experienced average processing times of approximately 38 seconds each, and three users experienced approximately 54 seconds each on the recommended two-core hardware. For production deployments with more than three expected concurrent users, a multi-worker deployment using multiple Streamlit server processes behind a load balancer is recommended.

Appendix M: Security Assessment

This appendix presents a structured security assessment of the AI Driven Business Sales Forecasting System, covering the key attack surfaces relevant to a web-based data upload and analysis application. The assessment follows a threat modelling approach adapted from the STRIDE framework, identifying potential threats and the mitigations implemented or recommended.

N.1 STRIDE Threat Model

STRIDE is a threat modelling framework developed by Microsoft that categorises threats into six types: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. The following analysis applies each category to the current system.

N.1.1 Spoofing

Spoofing threats in this context relate to the possibility of an attacker impersonating a legitimate user to access their analysis session or results. In the current single-user session design, there are no user accounts, so traditional credential-based spoofing is not a concern. However, if the system is deployed with multi-user authentication (as recommended in Future Work section 8.2.3), standard authentication threats such as credential stuffing, session token theft, and phishing would become relevant. The recommended mitigation for future multi-user deployment is to use a well-established identity provider such as OAuth 2.0 with Google or Microsoft accounts rather than implementing custom authentication.

N.1.2 Tampering

The primary tampering concern is malicious file upload: an attacker could attempt to upload a crafted file designed to exploit vulnerabilities in the file parsing libraries (pandas, openpyxl) rather than legitimate sales data. The system mitigates this risk by validating the file structure against the expected schema before passing it to the processing pipeline, and by running file parsing in the same process as the application rather than in an isolated sandbox. For higher-security deployments, running file parsing in a separate sandboxed process would provide stronger isolation.

A second tampering concern is manipulation of the session state. Streamlit session state is stored server-side and is not directly accessible to users, which provides reasonable protection against client-side state tampering. The session identifier is managed by Streamlit's session management infrastructure.

N.1.3 Information Disclosure

Information disclosure is the most significant security concern for this application given that it processes commercially sensitive business data. The primary mitigations are the session-scoped in-memory data model (data is not persisted beyond the session) and the recommendation to deploy over HTTPS only. The system does not log or transmit uploaded data to any external service.

One information disclosure risk that deserves attention in production deployments is the server error log. If unhandled exceptions occur during processing, they may include fragments of the uploaded data in the stack trace. The application implements a global exception handler that catches processing errors and displays user-friendly error messages while suppressing raw stack trace output in the production configuration.

N.1.4 Denial of Service

Denial of service attacks against the application could take the form of large file uploads designed to exhaust server memory, or repeated concurrent requests designed to saturate CPU resources. The system implements file size limits configurable through `config.py` to prevent excessively large uploads. Row count limits provide a secondary defence against files that are small on disk but expand to large DataFrames in memory (for example, highly compressed CSV files).

Rate limiting is not implemented in the current version. For production deployments exposed to the public internet, rate limiting at the network level (for example, using Nginx or a cloud load balancer) is recommended to prevent automated abuse.

N.1.5 Elevation of Privilege

Elevation of privilege is not a significant concern in the current single-user, session-scoped design. The application runs as a non-root system user in the recommended deployment configuration. The Python runtime and Streamlit framework do not execute any user-provided code, which eliminates the primary vector for privilege escalation in web application contexts (remote code execution through code injection).

N.2 Dependency Security

All third-party dependencies should be pinned to specific versions in the `requirements.txt` file to prevent silent upgrades that could introduce security vulnerabilities. The project maintains a pinned requirements file and recommends running `pip-audit` or `safety check` as part of the deployment pipeline to scan for known vulnerabilities in dependencies.

The most security-sensitive dependencies from a data parsing perspective are `pandas` and `openpyxl`, both of which have had historical vulnerabilities related to parsing malformed files. Maintaining up-to-date versions of these libraries and monitoring their security advisories is recommended.

Appendix N: Market Analysis and Business Case

This appendix provides a brief market analysis contextualising the AI Driven Business Sales Forecasting System within the broader retail analytics software market, and presents an indicative business case for the system's commercial viability.

O.1 Market Size and Opportunity

The global retail analytics market was valued at approximately USD 5.8 billion in 2023 and is projected to grow at a compound annual growth rate of 18.4 percent through 2028, driven by increasing adoption of digital point-of-sale systems, growing awareness of data-driven decision-making, and falling costs of cloud computing infrastructure. The small and medium enterprise segment represents a significantly underserved portion of this market.

Pakistan's retail sector is estimated to generate annual revenues exceeding USD 152 billion, making it one of the largest retail markets in South Asia. However, the overwhelming majority of Pakistani retail businesses are small or micro-enterprises operating with minimal analytical infrastructure. The combination of rapidly increasing POS adoption, growing mobile payment penetration, and the absence of affordable AI-powered analytics tools creates a compelling market opportunity for a system of this type.

At the regional level, similar market dynamics apply across Bangladesh, Sri Lanka, Nepal, and the informal retail sectors of India, suggesting that a system designed for the Pakistani SME market could be extended to a significantly larger addressable market with relatively modest localization effort.

O.2 Competitive Positioning

As established in the literature review in Chapter 2, the proposed system occupies a unique position in the market landscape by combining three capabilities that are not currently available together in a single accessible tool: advanced machine learning forecasting, co-purchase recommendation, and a completely non-technical data-upload-driven interface.

Competitive Dimension	Assessment vs Existing Tools
ML forecasting accuracy	Competitive with enterprise tools; superior to basic BI (Power BI, Tableau)
Recommendation capability	Unique in the accessible SME tool category
Technical barrier to entry	Zero (upload and go); significantly lower than Python libraries
Cost of deployment	Free on Streamlit Community Cloud; lowest in category
Dataset format flexibility	CSV and XLSX; comparable to spreadsheet tools
Scalability	Up to 100K rows; sufficient for SME use cases
Customisation for large enterprises	Limited; not designed for enterprise-scale multi-store deployment

Table 244: Competitive Positioning Assessment

O.3 Indicative Business Case

To illustrate the potential value the system could deliver to a typical user, consider a grocery retail business with annual revenue of PKR 15 million and a product catalog of 200 items. Based on the literature on the impact of inventory optimization on retail performance, a 10 percent improvement in inventory decision accuracy translates to approximately 2 to 4 percent reduction in stockout-related lost sales and 3 to 5 percent reduction in overstock carrying costs.

For a business of the described scale, these improvements represent a combined annual saving of approximately PKR 300,000 to 570,000 in lost sales and carrying costs. Against a deployment cost of zero (Streamlit Community Cloud) or approximately PKR 50,000 per year for a dedicated cloud VM, the return on investment is strongly positive within the first year of adoption, even under conservative assumptions.

The recommendation engine adds a further source of value through cross-selling uplift. If product recommendations lead even 5 percent of transactions to include one additional recommended item, and the average additional item value is PKR 200, then for a business processing 200 transactions per day, the daily revenue uplift is PKR 2,000, or approximately PKR 730,000 per year.

O.4 Path to Commercialisation

While the current system is developed as an academic project, the technical foundation is sufficiently mature to serve as the basis for a commercially deployable product with the following additional development investment. The estimated effort figures are indicative and based on the team's experience of the development work completed.

Development Area	Description and Estimated Additional Effort
Multi-user authentication	OAuth 2.0 integration with Google or Microsoft; estimated 2 weeks
Persistent data storage	PostgreSQL integration for saving analyses across sessions; estimated 3 weeks
Real-time POS integration	REST API connectors for common Pakistani POS systems; estimated 6 weeks
Automated model retraining	Scheduled retraining pipeline with MLflow experiment tracking; estimated 4 weeks
Urdu language localisation	UI translation and right-to-left layout support; estimated 3 weeks
Mobile responsive design	Progressive web app optimisation for smartphone browsers; estimated 2 weeks
Customer support and documentation	Help centre, onboarding tour, video tutorials; estimated 4 weeks
Security audit	Third-party penetration test and vulnerability remediation; estimated 3 weeks

Table 255: Path to Commercialisation - Additional Development Areas

Appendix P: Detailed Description of System Output Screens

This appendix provides detailed textual descriptions of each major screen and output panel of the AI Driven Business Sales Forecasting System, serving as a substitute for screenshot illustrations. The descriptions are written to enable a reader to fully understand the visual design and information architecture of the system without access to the running application.

P.1 Application Home Screen

Upon navigating to the application URL, the user is presented with a clean white page with a left sidebar and a main content area. The page title, AI Driven Business Sales Forecasting System, is displayed in a large bold font at the top of the main content area, followed by a two-sentence description explaining that the system accepts uploaded store data and generates AI-powered sales forecasts and product recommendations.

Below the title and description is the file upload widget, rendered as a dashed rectangular box with a cloud upload icon and the text Drag and drop your CSV or Excel file here, or click to browse. Below the box is a smaller text note indicating the accepted file formats and the maximum file size. The sidebar contains a collapsed information panel with system version information and a link to the user guide.

P.2 Data Preview Panel

After a file is successfully uploaded, a data preview panel appears below the upload widget. The panel is headed by a green success banner reading Data loaded successfully followed by a summary line showing the number of rows and columns detected. Below this is a scrollable table showing the first ten rows of the uploaded dataset with all original column names preserved. Column headers are bold and the table has alternating row shading for readability.

A column mapping summary appears below the preview table, showing which uploaded column names were matched to the system's expected field names. Any columns that were not recognized are listed separately in an amber information box, informing the user that those columns will be ignored during analysis. This transparency allows users to identify and correct column naming issues before proceeding.

P.3 KPI Dashboard

The KPI dashboard occupies the full width of the main content area and is organized as a three-column grid of metric cards. Each card has a light blue or light green background, a metric label in small grey text at the top, and the metric value in large bold black text in the center. A thin coloured bottom border on each card provides additional visual distinction.

The seven KPI cards are arranged as follows: Row 1 contains Total Revenue, Total Cost, and Total Profit. Row 2 contains Profit Margin, Total Transactions, and Unique Products. Row 3 contains Best Month (spanning the full width). Below the KPI cards is a horizontal divider followed by the monthly profit trend chart, which is a Plotly line chart with the month on the x-axis and total profit on the y-axis. Hovering over any data point reveals a tooltip showing the exact month label and profit value.

P.4 Forecasting Results Panel

The forecasting results panel is headed by a section title and a brief one-sentence explanation of what the predictions represent and what model generated them. Below the

explanation are two tables displayed side by side using Streamlit's column layout. The left table is headed Top 5 Most Profitable Products (Next Month Forecast) and has a green header row. The right table is headed Top 5 Least Profitable Products (Next Month Forecast) and has an amber header row.

Each table has four columns: Rank, Product Name, Predicted Profit (PKR), and Predicted Quantity. Rows are sorted by predicted profit in descending order for the top-five table and ascending order for the bottom-five table. The Rank column displays bold rank numbers. Below each table is a small text note reminding the user that predictions represent model forecasts based on historical data and should be used as planning inputs rather than guarantees.

Below the two forecast tables is a horizontal bar chart showing predicted next-month profit for all products in the dataset, not just the top and bottom five. This chart allows users to see the full distribution of predicted product performance and identify clusters of products at similar profitability levels. The top five and bottom five products are highlighted in contrasting colours (green and amber respectively) while the remaining products are shown in grey.

P.5 Recommendation Panel

The recommendation panel is the fourth major section of the interface. It is headed by a section title and a brief explanation describing the recommendation model as an association rule mining engine that identifies products most frequently purchased together with the selected product.

The product selector is a Streamlit selectbox dropdown with a search input field. Typing partial product names filters the dropdown list dynamically. Once a product is selected, a table of up to five recommendations appears below the selector within approximately one to two seconds. The recommendation table has three columns: Rank, Recommended Product, and Co-occurrence Frequency. Each row is formatted with the rank number, the recommended product name, and an integer count showing how many transactions in the uploaded dataset contained both the query product and the recommended product.

If the selected product does not appear in sufficient transactions to generate reliable recommendations (fewer than the minimum support threshold), an informational message is displayed in place of the recommendation table, reading: Insufficient transaction history for reliable recommendations for this product. Please select a product with higher sales volume.

P.6 Export and Download Section

The final section of the interface contains two download buttons. The first button, labelled Download Full Forecast CSV, triggers a download of a CSV file containing the next-month profit and quantity predictions for every product in the uploaded dataset. The file is named with the format forecast_YYYY-MM-DD.csv using the current date.

The second button, labelled Download KPI Summary CSV, triggers a download of a single-row CSV file containing all seven KPI values computed for the uploaded dataset. This file is intended for import into business reporting tools or spreadsheets where the KPI values can be tracked over time as the user repeats the analysis with updated datasets. Both download buttons are styled in a dark blue colour consistent with the overall colour scheme of the application.

Appendix Q: Individual Contribution Statement

This appendix provides a detailed breakdown of the individual contributions of each team member to the project, in accordance with the University of South Asia Department of Computer Science requirement for group projects. Both team members contributed equally to the overall project, and all major design and implementation decisions were made jointly. The division below reflects primary responsibility for specific components rather than exclusive authorship.

Q.1 Student 1 Contributions

Component / Activity	Description of Contribution
Data ingestion module	Primary author of <code>data_ingester.py</code> including file type detection, schema validation, and column alias matching.
Feature engineering pipeline	Primary author of <code>feature_engineer.py</code> including all lag, rolling, and temporal feature generation logic.
XGBoost model training	Conducted hyperparameter grid search; primary author of <code>forecasting_model.py</code> ; responsible for model evaluation and accuracy reporting.
Forecasting results UI	Implemented the forecast results panel including the top-5 and bottom-5 tables and the full-product bar chart.
System integration testing	Designed and executed integration test scenarios; documented results in Chapter 6.
Chapters 5 and 7	Primary author of Implementation and Results chapters of this documentation.

Table 266: Student 1 Primary Contributions

Q.2 Student 2 Contributions

Component / Activity	Description of Contribution
Data preprocessing module	Primary author of <code>data_preprocessor.py</code> including missing value handling, duplicate removal, and outlier treatment.
KPI calculation module	Primary author of <code>kpi_calculator.py</code> including all seven KPI computations and the monthly trend aggregation.
Apriori recommendation engine	Primary author of <code>recommendation_model.py</code> including transaction matrix construction, rule mining, and recommendation retrieval.
KPI dashboard and recommendation UI	Implemented KPI card layout, monthly trend chart, and the recommendation panel with product search.

User acceptance testing	Designed UAT protocol; recruited and facilitated five UAT sessions; analysed and documented results.
Chapters 1, 2, 3, and 8	Primary author of Introduction, Literature Review, Requirements, and Conclusion chapters of this documentation.

Table 277: Student 2 Primary Contributions

Q.3 Joint Contributions

The following activities were conducted jointly by both team members throughout the project:

- Requirements elicitation and use case definition (Chapter 3)
- System architecture design and 4+1 view modelling (Chapter 4)
- Database schema design and output format specification
- Technology stack selection and justification
- Code review and pair programming sessions at key integration points
- Project planning, milestone tracking, and advisor meeting preparation
- Documentation review, proofreading, and final formatting
- Project presentation preparation and delivery

Q.4 Advisor Supervision Log

The following table summarises the formal supervision meetings held with the project advisor throughout the project period. Additional informal communication was conducted via email and the university learning management system.

Appendix R: Version History and Change Log

This appendix documents the version history of the AI Driven Business Sales Forecasting System.

R.1 Development Version History

v0.1.0 - November 2024 (Alpha): Established foundational data ingestion pipeline supporting CSV file uploads only. Basic schema validation implemented. User-facing output limited to data preview table. Reviewed and approved by project advisor as satisfactory Iteration 1 deliverable.

v0.2.0 - December 2024 (Alpha): Added Excel upload support. Completed full data preprocessing pipeline: missing value imputation, duplicate removal, outlier treatment, and date standardisation. Implemented KPI calculator with all seven dashboard metrics. Established Streamlit UI with KPI card layout and monthly trend chart. Advisor review approved with feedback on KPI card colour contrast.

v0.3.0 - December 2024 (Beta): Integrated feature engineering pipeline including temporal, lag, and rolling statistical features. Integrated initial XGBoost forecasting model achieving R-squared of 0.76. Added top-five and bottom-five forecast tables to UI. Advisor review: accuracy improvement to 80 percent required before Iteration 4.

v0.3.1 - January 2025 (Beta Patch): Added sine-cosine month encoding to temporal features. Introduced product-level target encoding for high-cardinality product identifier column. Added rolling standard deviation features. Model accuracy improved to R-squared of 0.81. Submitted for advisor review.

v0.3.2 - February 2025 (Beta Patch): Extended hyperparameter grid search. Final model accuracy R-squared of 0.912 achieved, exceeding the 80 percent target. Added model evaluation metrics panel to the UI for forecast transparency. Advisor approved; team authorized to proceed to Iteration 4.

v0.4.0 - March 2025 (Release Candidate): Added Apriori association rule mining recommendation engine. Implemented recommendation panel with product search dropdown. Added full-product horizontal bar chart for forecast visualization. Implemented CSV export for forecast results and KPI summaries. Finalized custom CSS styling for KPI cards and section dividers.

v1.0.0 - April 2025 (Final Release): Addressed all UAT feedback: partial-match product search, informational tooltips on KPI cards, improved cold-start handling for low-transaction products. Final documentation completed. This version submitted for academic assessment.

R.2 Known Limitations in v1.0.0

- Multi-currency datasets are not supported. All monetary values are assumed to be in Pakistani Rupees.
- Inconsistent date formats within a single file may cause incorrect date parsing for a subset of rows.
- Minor spelling variations in product names are treated as different products, potentially splitting transaction history.

- Incremental data updates are not supported. Each session requires a complete historical dataset upload.
- Session data held in server memory is not encrypted at rest during an active session.
- The recommendation engine does not generate suggestions for products appearing in fewer transactions than the minimum support threshold.

Appendix S: Frequently Asked Questions

This appendix presents answers to questions raised during user acceptance testing sessions and those anticipated to arise frequently among business users of the system.

S.1 General and Setup Questions

Q: Do I need to install anything to use the system?

A: No. The system runs entirely in your web browser. Navigate to the application URL, upload your data file, and the analysis begins automatically. No software installation, account creation, or technical setup is required.

Q: Is my business data safe?

A: Your data is processed within your session only and is not saved to any database or shared with any third party. When you close your browser or the session expires, all data is automatically removed from the system. Always access the system over a secure HTTPS connection for maximum protection.

Q: What file formats does the system accept?

A: The system accepts both CSV files and Excel files in the .xlsx format. Files must contain the required columns described in Appendix A. A clear error message is displayed if the uploaded file does not meet the requirements.

Q: How much historical data do I need?

A: A minimum of approximately 1,000 transactions covering at least six months of sales history is recommended for reliable forecasts. More data generally improves accuracy. Products with fewer than the minimum transaction threshold will not have forecasts or recommendations generated.

Q: Can I use the system on my mobile phone?

A: The system is accessible on mobile browsers, though the experience is optimized for desktop screens. On smaller screens, the two-column forecast tables may require horizontal scrolling. A mobile-optimized version is identified as a future development priority in Chapter 8.

S.2 Forecasting and Recommendation Questions

Q: How accurate are the sales forecasts?

A: The forecasting model achieves approximately 91 percent accuracy on historical test data, meaning that predicted values are within 17 percent of actual values on average. Forecasts are most reliable for products with long and stable sales histories. Always combine AI forecasts with your own knowledge of upcoming events and promotions.

Q: Can the system forecast a new product with no history?

A: No. The system relies on historical patterns for each product. Products with no transaction history cannot be forecast. Future versions may support similarity-based forecasting from related products as described in Chapter 8.

Q: Why does the system only show 5 products in each forecast table?

A: The top-5 and bottom-5 products are shown as the primary view because these are most likely to require management attention. The full forecast for all products is available by downloading the forecast CSV file using the export button at the bottom of the forecasting section.

Q: How often should I run the analysis?

A: Running the analysis monthly is recommended for most businesses. As you accumulate more recent transaction data, re-uploading the updated complete dataset will produce more current forecasts. Businesses with highly seasonal or fast-moving inventory may benefit from more frequent analysis.

Q: Why does a product show no recommendations?

A: The recommendation engine requires a product to appear in a minimum proportion of transactions to generate reliable co-purchase associations. Products with very low sales volume fall below this threshold, and the system displays an informational message rather than unreliable recommendations.

Q: Can I change which products are selected for forecasting?

A: The current version automatically selects the top-5 and bottom-5 products based on predicted profit. The full prediction table (available via CSV download) allows you to review forecasts for all products and manually select those most relevant to your decisions.